

BREAST-CANCER PROGNOSIS: LEVERAGING CANCER REGISTRY DATA FOR
SURVIVAL PREDICTION

A Project

Presented

to the Faculty of

California State University, Dominguez Hills

In Partial Fulfillment

of the Requirements for the Degree

Master of Science

in

Computer Science

by

Pranathi Deepak

Fall 2025

PROJECT: BREAST-CANCER PROGNOSIS: LEVERAGING CANCER REGISTRY DATA
FOR SURVIVAL PREDICTION

AUTHOR: Pranathi Deepak

APPROVED:

Liudong Zuo, Ph.D.
Project Committee Chair

Jianchao Han, Ph.D.
Project Committee Chair

Pei-Chen Peng, Ph.D.
Committee Member

ACKNOWLEDGEMENTS

I want to express my gratitude to Dr. Zuo, whose advice, support, and valuable feedback influenced the course and quality of this effort. Throughout this entire study process, his patience and dedication to student learning gave me the clarity I needed.

I am sincerely thankful to Dr. Peng for sharing her expertise and clinical insights, which broadened my understanding of breast cancer prognosis and greatly strengthened the relevance of this work.

Additionally, I want to express my gratitude to the CSUDH Computer Science teachers and staff for fostering an atmosphere that encourages inquiry, teamwork, and intellectual development.

My sincere gratitude is extended to my family and close friends, whose unwavering encouragement, positive words, and faith in my abilities kept me inspired over long evenings spent coding, editing, and experimenting. Their support provided me the self-assurance to finish my endeavor.

TABLE OF CONTENTS

ACKNOWLEDGEMENTS	iii
TABLE OF CONTENTS.....	iv
LIST OF TABLES	vi
LIST OF FIGURES	vii
ABSTRACT.....	
1. INTRODUCTION	1
Background And Motivation.....	2
Problem Statement	3
Reasearch Objectives	3
Scope of the Project.....	4
Contributions to this work.....	4
Key Definitions	5
2. LITERATURE REVIEW	7
SEER Database and its role in Survival Modeling	8
Model Evaluation Metrics	8
Statistical Models in Survival Analysis	9
Machine Learning Models for Surival Predictions	9
Ensemble Learning in Survival Prediction.....	10
3. DATA SOURCE AND COHORT CONSTRUCTION.....	11
Cohort Selection and Data Extraction	11
Data Processing Pipeline	12
Final Feature Set.....	13
4. DATA PREPROCESSING AND FEATURE SIGNIFICANCE	15
Data Cleaning and Quality Control.....	15
Feature Encoding and Transformation	16
Feature Scaling and Model Compatibility	17
Feature Significance	17
5. EXPLORATORY DATA ANALYSIS (EDA)	19
Cohort Level Survival Characteristics	19
Distribution of Key Demographic Variables	20
Tumor Characteristics and Disease Severity	21
Treatment Related Variables	22

Survival Stratification by Clinical Variables	22
Correlation of Feature Interaction Analysis	23
Summary	24
6. SURVIVAL MODELING	25
COX Proportional Hazards Model	25
Random Survival Forest	28
DeepSurv: Deep Learning Based Survival Modeling	31
Comparative Analysis and Ensemble Modeling	33
7. EVALUATION FRAMEWORK AND RESULT ANALYSIS	35
Primary Performance Results: Discrimination and Stability	35
Time-Dependent Discrimination Analysis	38
Risk Stratification and Survival Curve Analysis	40
Model Interpretability and Explainability using SHAP	41
8. CONCLUSION	46
REFERENCES	47
APPENDIX A: SOURCE CODE	48

LIST OF TABLES

1. Predictor Variables.....	12
2. Bootstrap C-index with 95% Confidence Intervals	36
3. Test-Set Concordance Index Across Models	37

LIST OF FIGURES

1. KM Curve for overall survival probability	20
2. Distributions of age group, race, and marital status.....	21
3. Distribution of tumor size and AJCC staging categories.....	21
4. Frequency distribution of treatment modalities	22
5. KM Curves by Cox model risk groups & summary stage	23
6. Correlation Heatmap	24
7. Time-Dependent AUC for Cox Proportional Hazards Model	38
8. Time-Dependent AUC for Random Survival Forest	39
9. SHAP Summary Dot Plot	42
10. SHAP Summary Bar Plot.....	42
11. SHAP Dependence Plot — Tumor Size	43
12. SHAP Dependence Plot — AJCC N Stage.....	43
13. SHAP Dependence Plot — Year of Diagnosis	44
14. DeepSurv Feature Interaction Heatmap (SHAP-Based Approximation)	45

ABSTRACT

The study investigates the survival outcomes of breast cancer using statistical and machine learning techniques applied to SEER registry data. The combination of Random Survival Forests, Cox Proportional Hazards modeling, and Kaplan-Meier estimators helped in capturing both linear and nonlinear risk patterns. A DeepSurv neural network is also employed to improve performance by leveraging representation learning for better predictions. Major preprocessing on the dataset involves survival-specific encoding, handling missing values, and modifying features. Concordance indices along with survival probability curves are used to evaluate the performances of these models, which allow comparison between traditional and modern survival strategies. The result is a framework where patient-specific risk can be determined, and key prognostic variables can be understood. This research has shown that integrating machine learning techniques with traditional survival models enhances the ability to evaluate complex clinical datasets and promote data-driven decision-making in oncology.

CHAPTER 1

INTRODUCTION

Breast cancer is still one of the most important health problems for women around the world and among the leading causes of cancer-related morbidity and mortality. Even with improvements in diagnosis and treatment, prognostication of long-term survival for a patient remains challenging given the heterogeneous nature of tumor biology, disease behavior, as well as individual patient-related clinical variables. Consequently, clinicians have become more dependent on data-derived computational tools to assist in prognosis assessment and therapeutic planning. In fact, survival analysis provides an effective platform for analyzing time-to-event endpoints like relapse or death and yields information beyond conventional methods.

Over the past few years, prognostication of survival has been largely enhanced as large population-based cancer registries such as the Surveillance, Epidemiology, and End Results (SEER) database have expanded. Specifics of demographics, pathology and type of treatment in these databases are of high quality for different patient subgroups which guarantees a good-performance model running. But the identification of clinically relevant patterns in such heterogeneous high-dimensional data, to be used as prognostic factors, needs analysis that are robust for truncation and can enable nonlinear interactions and complex risk relations. The expanding field of machine learning provides inspiring new approaches that enhance the classical statistical models to predict better and interpret results.

This study investigates the extent to which the contemporary survival analysis techniques namely Cox Proportional Hazards (Cox PH), Random Survival Forests (RSF) and a deep learning-based model, DeepSurv can be used for more precise estimation of individual breast cancer survival. Although Cox PH is still the most common model in clinical studies because it

has an interpretable form, machine learning and neural network models are superior to Cox PH for nonlinear and complex variables patterns. Through side-by-side comparison and incorporation in an ensemble framework, this study aims to offer a more comprehensive insight at the patient-level risk.

In addition to model building, this effort also focuses on constructing a working prediction pipeline that can ultimately be adopted by clinicians or researchers into decision support systems. The main goal was to establish a reproducible, transparent workflow (from raw data preprocessing up to prediction) that complies with what is generally expected in real-world medical analytics. While this analysis does not replace the clinical judgment, they help in refining or making model/algorithm-driven observations that can provide insights into patterns of prognosis.

Background and Motivation

Survival estimation is an important part of cancer care. Precise estimates of expected outcomes are crucial for oncologists to adequately stratify treatment plans, consider potential benefits from therapies, and communicate risk to patients. Conventional methods for survival analysis, such as Kaplan-Meier curves or Cox PH models, offer a basic understanding but are limited when relationships with data become deeply nonlinear, and in this case, we have many categorical variables, as seen in the SEER databases.

Thanks to machine learning techniques, these limitations have been overcome. Random Survival Forests approximate complex interactions without imposing harsh proportional hazards assumptions. DeepSurv, the neural network version of Cox PH, is more flexible, not only for learning nonlinear risk functions but also for transparency by preserving hazard ratios. When used carefully, these methods can reveal the subtle structure that classical models miss.

This project is driven by the requirement for cross-validation and cross-technique comparison among these different methods of analysis. By combining the statistic, tree-based, and deep learning models, this study seeks to find which methods give the best and most reliable prognostic information and how these could potentially be used together.

Problem Statement

Predicting the prognosis in breast cancer is challenging because of the following factors:

- Great diversity among tumor features and patient characteristics
- Interactions between biological and treatment factors that are complex
- The effect of censored samples in survival data
- Limitations of single-model forecasting approaches

Traditional models often assume linear or even simpler relationships that are not realistic with the data generating process. Machine Learning models can improve their performance, but require special attention about the preprocessing of data, feature encoding and interpretability. The primary problem addressed in this project is the development and analysis of a multi-model survival prediction framework, which enhances individualized risk assessment based on SEER breast cancer data.

Research Objectives

This project focuses on four primary objectives:

1. Develop and preprocess a structured survival dataset from SEER breast cancer data, including engineered characteristics such as age at diagnosis, tumor, and treatment features.
2. Train and compare a variety of survival analysis models, Cox PH, Random Survival Forests and Deepsurv, with the same preprocessing pipelines.

3. Compare models with concordance index (C-index), survival curves and feature importance.
4. Construct an ensemble risk-scoring pipeline to make an integrated survival prediction for a new patient using normalized risks from all four models.

Their respective realization yields not only methodological insights, but also a working prediction interface for approximation of actual clinical data scenarios.

Scope of the Project

The scope of this study includes:

- SEER registry data from 2010 to 2017 for women who were diagnosed with breast cancer.
- 22 clinically relevant features for modeling.
- Preprocessing categorical, ordinal, and numerical features.
- Utilizing classical, machine learning, and deep learning models to predict survival analysis.
- Evaluating models using solid metrics and visual diagnostics.
- Building an ensemble approach to combine model predictions.

The project is not a study of treatment optimization, real-time clinical application, or causal inference on the effect of therapies; rather, the models provide cause-specific survival probabilities.

Contributions to this Work

The major contributions of this project include:

- A general survival model framework for comparing statistical, ML, and deep learning models.

- A trained and validated DeepSurv neural network using custom SEER data preprocessing and loss functions.
- A combined coefficient/permutation score/SHAP feature importance analysis.
- An ensemble risk pipeline integrating the strengths of models into a universal normalized risk score.
- A workflow package that can potentially be built into a clinical-facing app, a Streamlit survival prediction tool.

These contributions advance our knowledge in survival modeling trade-offs and practical methods for working with large medical datasets.

Key Definitions

To ensure clarity throughout this report, the following essential survival analysis terms are defined:

- **Event:** The outcome of interest (e.g., death due to cancer).
- **Censoring:** This is defined as an unknown event time for a patient (e.g., lost to follow-up or still alive at last contact).
- **Survival Time:** The time from the diagnosis of an event or to censoring.
- **Hazard Ratio (HR):** A way to measure how a predictor influences the hazard; $HR > 1$ suggests a greater risk.
- **Proportional Hazards Assumption:** An assumption of the Cox PH model, which states that the hazard ratios do not change over time.
- **Concordance Index (C-index):** Performance metric that quantifies how well the model ranks patients according to risk.

- **Random Survival Forest:** An ensemble of decision trees that works well with non-linearities.
- **DeepSurv:** A deep neural network which learns a nonlinear risk function as it trains under the Cox-style loss.

These definitions play an important role in interpreting the issues and the results given in later sections.

CHAPTER 2

LITERATURE REVIEW

In survival analysis, duration is the time until an event, such as death or disease progression occurs. One characteristic of such a type of data is censoring, meaning some patients do not experience the event before the end of the observational period. For this reason, traditional regression techniques are not suitable, and specific survival models have been developed.

In healthcare, such a survival analysis is used to make prognosis prediction, risk stratification, and analyze the impact of clinical factors on patient outcomes. Breast cancer research depends largely on these techniques, given that survival differs greatly based on tumor features, stage of diagnosis, and treatment scenario.

Classical methods such as the Kaplan–Meier (KM) estimator and Cox Proportional Hazards (Cox PH) model are commonly applied because of their interpretability and relevance to clinical management. But datasets today frequently include intricate, non-linear patterns that old models might not fully recognize.

To overcome this limitation, more recent machine learning-based survival models, such as RFS or DL approaches (DeepSurv), have emerged. Such methods consider interactions and non-linearities, which help in making better predictions. This work extends classical and modern methodologies to formulate a comparative and ensemble survival prediction framework for breast cancer with SEER registry data.

SEER Database and Its Role in Survival Modeling

Surveillance, Epidemiology, and End Results (SEER) Program is a population-based system that collects cancer incidence data from specific geographic areas covered by high-quality tumor registries throughout the United States. SEER data are of a high quality for patient demographics, tumor characteristics, treatment variables, and long-term survival. Therefore, it is particularly well-suited for studies that need complete and accurate data on time-to-event measurement. SEER has the stage of disease, hormone receptor status, and lymph-node involvement recorded in a consistent way for breast cancer prognosis. These capabilities facilitate robust survival modeling and reproducible analyses, thereby making SEER an indispensable platform for the development of the survival prediction models proposed herein.

Model Evaluation Metrics

Metrics like accuracy and AUC would not be appropriate to evaluate survival models that are sensitive to censored data and varying follow-up times. The Concordance Index, or C-index, would be the preferred metric because it would allow measurement of the predictive ability of the survival model based on its accuracy at ranking subjects based on risk. A C-index with a higher value would indicate that subjects who experience an event earlier would receive higher risk scores. As survival modeling is an estimation of times-to-events and not a classification task, it would be inappropriate to compare subjects based on ROC curves derived at different endpoints.

Calibration methods are also very relevant to topics related to survival calibration, indicating the extent to which predicted survival probabilities agree with what truly occurs. A common method within survival calibration relates to the usage of Kaplan-Meier curves and predicted survival curves. Some papers have also resorted to Brier scores, which evaluate global

prediction error and relate to accuracy as well as calibration. Both these methods for accuracy and calibration have been very useful because they offer a broader scope for accuracy and calibration assessment.

Statistical Models in Survival Analysis

Survival statistical models are the cornerstone of clinical prognostic research. The most popular methods are the Kaplan–Meier estimator and the Cox Proportional Hazards (Cox PH) model. The Kaplan–Meier model is a nonparametric estimate of survival probabilities over time and is widely used to compare groups, such as age groups or stage of tumors, through easily interpretable survival curves. Although descriptive, it does not include multiple predictors at the same time.

The Cox PH model is based on the concept that the covariates have a different effect on inherited risk at different times. It models the relative risk between groups (i.e., hazard ratio) as a time-invariant factor, and researchers can then examine how clinical and demographic factors predict survival. Its interpretability in terms of hazard ratios has rendered it fundamental for research in oncology. In this work, baseline group comparison is done by Kaplan–Meier analysis, and the Cox PH model acts as one of the basic predictive models to assess risk and contribute towards ensemble survival prediction methodology.

Machine Learning Models for Survival Predictions

Machine learning approaches have broadened the capacity to predict survival through the extraction of complex nonlinear patterns that might be missed by conventional models. Random Survival Forests (RSF) extend popular ensemble learning techniques to the setting where observations are subject to censorship. By forming several survival trees and combining their predictions, RSF effectively addresses high-dimensional features, non-linear interactions, and

missing data compared to classical methods. The fact that it can model individualized survival curves makes it an appropriate method for heterogeneous cancerous populations.

DeepSurv, a Cox-model extension based on neural networks, learns a non-linear risk function from the patient's features. Rather than making the proportional hazard assumption, it learns complex relationships via hidden layers and estimates personalized risk scores. This agnosticism allows DeepSurv to learn subtle patterns in tumor features, treatment details, and population characteristics that are associated with survival. In this work, RSF and DeepSurv are used to enrich the predictive power of the Cox model. Their results are combined in the ensemble study, which leads to robustness and reinforces the reliability of survival risk prediction.

Ensemble Learning in Survival Prediction

Ensemble learning can aggregate predictions from several models to enhance robustness and prediction accuracy, a property that is particularly valuable in survival analysis where patient heterogeneity and censoring often make single-model assumptions unreasonable. Approaches such as the Random Survival Forests (RSF) extend tree-based ensembles for time-to-event data and identify nonlinear effects and interactions that might be overlooked by standard Cox PH models. Each model brings its own unique strengths – interpretability, nonlinear learning, and high-dimensional modeling. Through score normalization and averaging, ensembles shrink the bias from individual models and hence often achieve superior discrimination measures, which are well-matched for survival prediction problems with modern causes of death.

CHAPTER 3

DATA SOURCE AND COHORT CONSTRUCTION

For this study, the SEER registry was the main database used to establish our analytic cohort. Rather than discussing SEER in general, this section describes how the dataset was operationalized for predicting survival. The SEER breast cancer datasets included information on demographics, tumor biology, stage of disease, treatment received, and follow-up on possible outcomes in standardized coding systems. These terms were necessary to define model inputs, such as age category (under 35, 36-59, or 60+), receptor status, lymph-node involvement, tumor size, and AJCC-7 staging. The month-level survival time and vital-status indicators of SEER made it possible to create reliable time-to-event labels. The completeness and consistency of these fields led to a high-quality dataset that could be used for the development and comparison of multiple survival models, such as Cox PH, Random Survival Forests, and DeepSurv.

Cohort Selection and Data Extraction

The original dataset was extracted from the SEER registry export, which included breast cancer patients diagnosed between 1990 and 2017. After import, all raw variables were checked, renamed, and formatted in a consistent manner across our modeling pipeline. The survival time was calculated using the variable for survival months, and the event indicator was generated from SEER vital status (coded as 1 for death and 0 for censoring).

Refinement of the cohort consisted of the exclusion of records with missing TNM staging (AJCC T, N, M), because this information is required for survival modelling. Tumor size that was represented in mixed textual format was converted to a numeric format by using regular expressions. Categorical covariates, including ER/PR/HER2 status, summary stage, type of surgery, and race, were included in the model for their high prognostic significance. Numerical

variables, including tumor size, the number of examined lymph nodes, and year of diagnosis, were retained in the dataset.

The final cleaned cohort was a clinically relevant and statistically valid dataset for modeling using Cox PH, Random Survival Forests, and DeepSurv, with an appropriate event rate post-stratified split.

Table 1 Predictor Variables

Feature Category	Variables Included	Clinical Meaning/ Rationale for Inclusion
Demographics	Age group, Sex, Race, Marital Status	Influence baseline risk, social support, and known disparities in breast cancer outcomes.
Diagnostic Factors	Year of diagnosis, Primary Site, Histology	Capture diagnostic era effects and tumor origin, affecting subtype classification.
Tumor Characteristics	Grade, Laterality, Tumor size, Summary Stage	Reflect tumor aggressiveness, disease extent, and biological behavior.
Node & Metastatic Status	Nodes examined, Nodes positive, AJCC T, AJCC N, AJCC M	Core indicators of tumor burden and metastatic spread, heavily influencing prognosis and treatment planning.
Receptor Status	ER Status, PR Status, HER2 Status	Key biomarkers guiding treatment and strongly predicting survival.
Treatment Variables	Surgery Type, Radiation, Chemotherapy	Capture therapeutic interventions that directly modify survival probability.

Data Processing Pipeline

The preprocessing technique has been standardized to prepare a dataset for modeling using the three modeling techniques. The missing numeric values like tumor size, nodes

examined, were replaced with the median, while the categorical attributes were replaced with the mode value.

In the Cox PH model, the categorical variables are one-hot encoded to result in a high-dimensional sparse matrix, which is filtered based on a certain threshold of variance, eliminating the need for the redundant encoded representations. The coefficients are normalized for the numeric covariates as well.

For the Random Survival Forest (RSF) analysis, categorical variables were encoded with integer codes, which is a requirement when working with tree models. Additionally, RSF-specific handling, such as memory safety during batch prediction, has been implemented to address large test datasets.

For DeepSurv, the goal of pure labeling (without one-hot encoding) is to prevent an explosion of dimensions. The numeric variables are scaled, and the inputs are converted to PyTorch tensors.

A stratified split between the training and testing sets ensured that the number of censored and uncensored patients remains Representative.

Final Feature Set

Ultimately, all three models combined cumulatively from a conceptual set of 22 clinical/demographic predictors, although encoded differently as per the requirement of the models. The shared set included age at diagnosis, gender, race, tumor factors (grade and size), tumor summary stage (either localized, advanced/regional, or distant), nodal/metastatic factors (T/N/M factors, nodes positive vs negative), and recipient factors (ER, PR, HER2), along with treatment factors (surgery, radiation, chemotherapy).

The Cox PH model handled high-dimensional representations of one-hot expanded inputs, whereas RSF and DeepSurv handled compact representations of the same inputs as label encoding. This ensured that the variations in the performance of the models were due to the rigor of the models and not due to how they were anchored to the data. The resulting feature matrix provided a comprehensive clinical view of patients with precise survival predictions.

CHAPTER 4

DATA PREPROCESSING AND FEATURE SIGNIFICANCE

Any real-world clinical survival analysis demands proper preprocessing to produce a model that is statistically valid, computationally stable, and clinically understandable. While the SEER database is one of the most valued resources for longitudinal cancer datasets, there are challenges in dealing with missing values, heterogeneous variable types, censored observations, and other issues in the data. Therefore, a well-defined preprocessing pipeline was developed to transform raw SEER data into input ready for any model, whether using traditional statistical models or newer machine learning techniques.

First, preprocessing for all survival models was carried out the same to ensure comparability. This pipeline included steps of data cleansing, generation of a survival outcome, feature selection, handling missing values, encoding, and scaling, all model specific. Emphasis was placed on maintaining clinically relevant variables and avoid unnecessary transformations that would harm interpretability. The resultant dataset served as a common base for the Cox Proportional Hazards, Random Survival Forest, and DeepSurv models.

Data Cleaning and Quality Control

The raw SEER data was initially processed for cleaning to make the data more reliable. Survival time data was extracted using the Survival months variable, which was then discretized into numeric form. Non-numeric or invalid survival times were omitted. A binary event variable was created using vital statistics, with patients who were deceased marked as event (1) and those who were still alive were censored or marked as 0. This is a widely recognized method of censoring in survival analysis that is more representative of people not at risk of an event, rather than being entirely lost from observation.

There were some missing or inconsistent data for several clinical variables. Missing data for numerical variables, such as tumor sizes and the number of lymph nodes, were replaced with the median value to minimize the effect of outliers. Missing data for categorical variables was replaced with the most frequent category. Missing data for AJCC classification of cancer (T, N, or M) were deleted since cancer stage is one of the important factors in determining patient survival in breast cancer.

Following that, stratified sampling was used to divide the data into a training data set and a testing data set. Both datasets were constructed using the event status. This ensured that the number of death observations and censored observations remained unbiased.

Feature Encoding and Transformation

The predictors included in our study were a combination of numerical, ordinal, and nominal clinical variables. Feature encoding methods were adopted to enable the developed model to be generalized with other survival models. In Cox Proportional Hazards modelling, to input the categorical variables as predictors of any stage recurrences and any distant metastases, we used one-hot encoding, which allows the linear model to independently estimate the hazard for each category. Numerical variables (tumor size, year of diagnosis, and number of lymph nodes) were standardized for better numerical optimization.

In the Random Survival Forest method, categorical variables were incorporated using integer codes and interpreted by tree-friendly algorithms that can capture nonlinear relationships/feature interactions without imposing any distributional assumptions. We utilized label encoding for the categorical features in DeepSurv as numerical inputs to assist efficient learning and avoid the memory overhead of high-dimensional one-hot representations. These

methods of encoding are specific to the model, ensuring computational efficiency as well as methodological consistency.

Feature Scaling and Model Compatibility

Feature scaling was crucial for training stability and a fair comparison of models. In models relying on gradient-based optimization, standardization was performed for continuous variables in Cox Proportional Hazards and DeepSurv. Besides that, having scaling prevented large numeric range features from affecting the model coefficients or neural network weights too much. In the case of Random Survival Forests based on recursive partitioning, scaling was not required, and raw encoded values were used.

One of the key decisions was to keep the feature sets identical across all the models. While preprocessing was specific to each model, the same underlying clinical variables were sustained for interpretability and comparability. This alignment ensured that any differences in the quality of modeling were emblematic of the ability of a model to model well, rather than differences in how data had been prepared for its application, thus increasing the defensibility around any comparison that had been made.

Feature Significance

Feature importance was determined through the interpretation methods relevant to each model, i.e., the magnitude of coefficients for Cox models, the permutation importance for Random Survival Forests, and SHAP values for DeepSurv. Irrespective of the methodology, factors that were associated with terms related to tumor burden, stage of cancer, lymph node involvement, and patient demographics, such as age and ethnicity, were consistently retained in the final models. This concordance between various modelling schemes supported the clinical significance of the final feature set.

A uniform pre-processing approach further facilitated the combination of several models into an ensemble risk model. Ensemble risk scores were also capable of leveraging the strengths of both statistical and machine learning models by harmonizing parameter definitions and source populations to achieve consistency between features as defined and survival outcomes. This approach allowed for strong risk stratification while preserving interpretability and methodological rigor.

CHAPTER 5

EXPLORATORY DATA ANALYSIS (EDA)

EDA is an explicit step in the survival analysis procedure that is intended to appreciate the characteristics of the data before formally embarking on building a survival model. In the context of breast cancer prognosis, EDA would be utilized to identify patterns with respect to patient characteristics, tumor information, treatment variables, and survival data. This step is also important for validating common assumptions in survival analysis, such as the proportional hazards assumption or risk stratification.

In this analysis, EDA on the cleaned SEER breast cancer data was conducted to explore distributions, correlations, and survival rates of the various clinical features. Patterns of censoring, event distributions, imbalances of features, or possible presence of bias were evaluated for the data using data summaries. The outcomes of these evaluation tools guided the preprocessing stage or modeling procedure.

Cohort Level Survival Characteristics

A fundamental principle of survival analysis is the understanding of overall survival in a population under consideration. The baseline survival probability over the test cohort was visualized using Kaplan–Meier estimation. The Kaplan–Meier curve showed a decrease in the expected survival probability, with a more pronounced drop observed among those with earlier follow-ups and tapering afterwards.

The percentage of right-censored observations (i.e., patients who were alive at the last follow-up) was high, as would be expected in clinical practice. This censoring scheme motivated the use of survival-focused modelling and discouraged the use of normal flow models, such as classification or regression. The Kaplan–Meier analysis also established a yardstick to which

model-based survival curves could be compared to evaluate whether the risk strata had been derived from the same population reflected by observed survival rates.

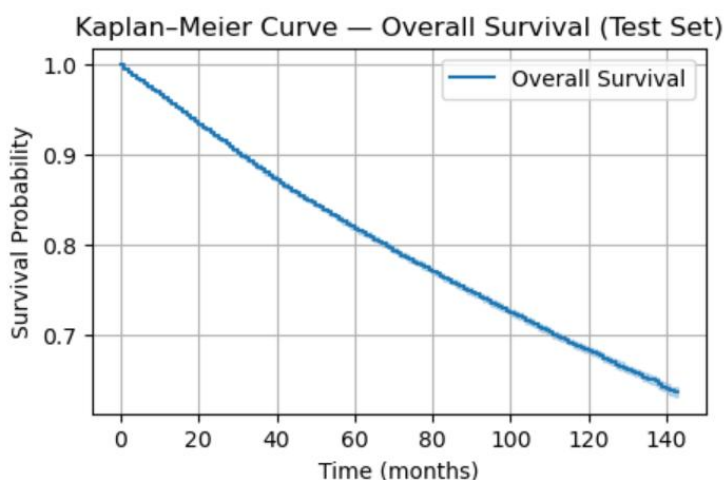


Figure 1: KM Curve for overall survival probability

Distribution of Key Demographic Variables

There were differences in the distribution of the population and disparities in survival across different age groups, genders, races, and marital statuses. The age composition consisted of more patients in the middle and older age groups, consistent with the commonly observed incidence of breast cancer. The cohort was predominantly female, as would be expected for a breast cancer cohort.

Distribution of race: There was some representation of Whites, Blacks, Asian/Pacific Islanders, and others. The distribution by marital status was also spread out and was kept as a factor variable since it is already known to influence the survival outcome in oncology research. A visual inspection of this distribution revealed some imbalances in the classes, which were corrected in modeling through stratification and regularization.

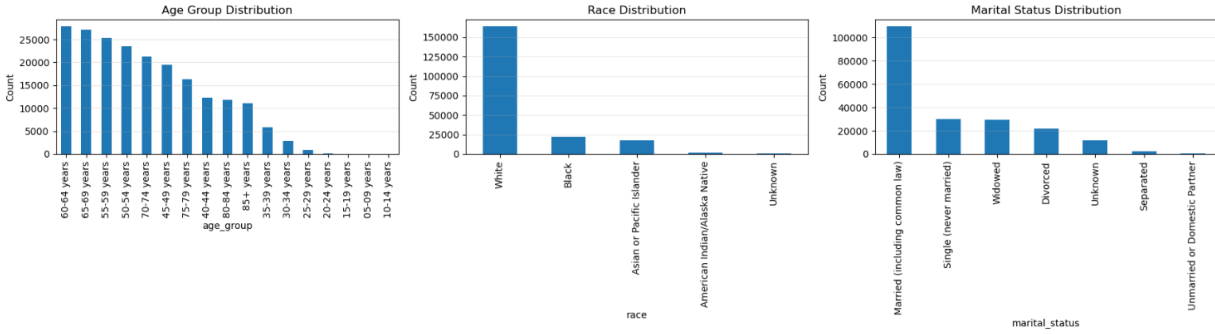


Figure 2: Distributions of age group, race, and marital status

Tumor Characteristics and Disease Severity

Tumor factors are important for the prediction of breast cancer survival. When examining the size of tumors and conducting an exploratory analysis, we found that the distribution was right-skewed, with most values at small sizes and fewer in extreme scales. This explained why median-based imputation and scaling in the preprocessing made sense.

Patients with AJCC T, N, and M stage information ranged widely through the various stages of disease. Although less common, the patients with more advanced T and N categories and metastatic disease were clinically relevant. In addition, summary stage further stratified the extent of disease into localized, regional, and distant categories for clinically staged cases, allowing for successful comparison of survival.

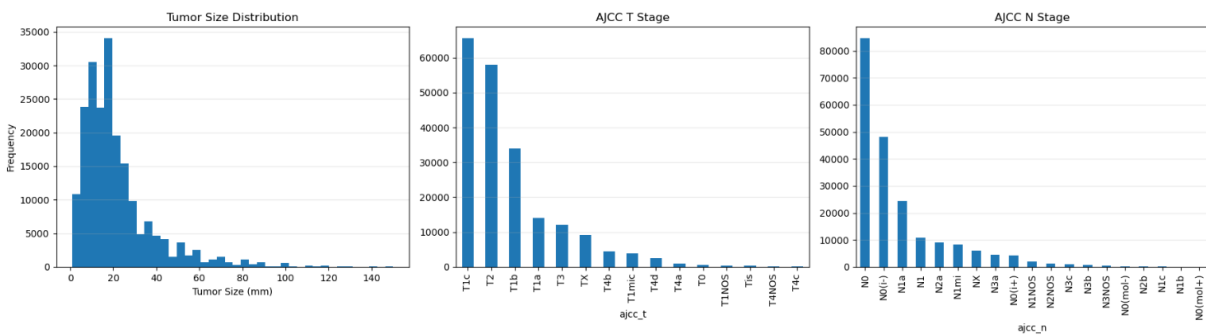


Figure 3: Distribution of tumor size and AJCC staging categories

Treatment Related Variables

Treatment factors, such as the type of surgery, use of radiation therapy (RT), and chemotherapy (CT), among others, were analyzed to explore the patterns of care in the cohort. There was substantial heterogeneity in surgical type, which may correspond to the differences in disease stage and therapeutic intent. Both radiation and chemotherapy variables were conditioned on the use of categorical indicators, which varied substantially across patients.

The distribution of hormone receptor status (ER, PR, and HER2) were clinically relevant, supporting their identification as prognostic variables. They later proved to challenge model predictions in machine learning-based techniques. The exploratory data analysis (EDA) corroborated the presence of notable variability in treatment-related parameters for use as covariates in survival modeling.

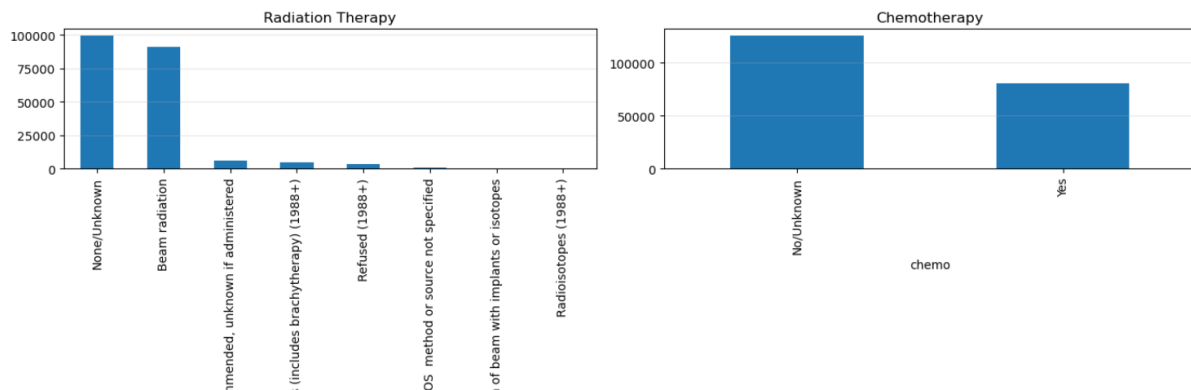


Figure 4: Frequency distribution of treatment modalities

Survival Stratification by Clinical Variables

To evaluate the impact of individual variables on survival, Kaplan-Meier curves were plotted for specific clinical parameters. Factors with significant separation of survival curves, including summary stage, AJCC N stage, and type of surgery, proved to be good predictors.

There was also a significantly better survival probability for patients with localized disease or fewer positive lymph nodes than those in an advanced stage.

Likewise, segregation of one-year survival curves amongst the model-specific risk groups (low-medium-high risk) was monotonic, confirming that the trained models could discriminate well. These plots yielded early insights into the fact that clinically meaningful patterns of risk were being learned by both conventional and learning models.

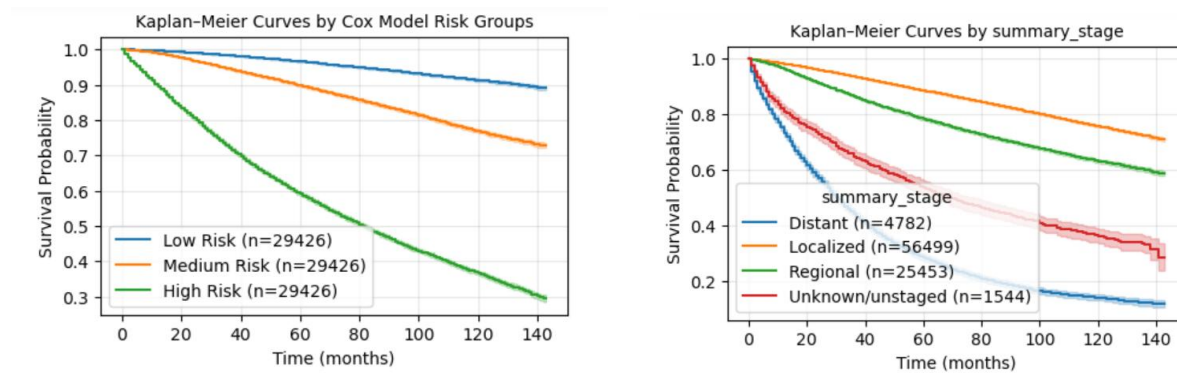


Figure 5: KM Curves by Cox model risk groups & summary stage

Correlation of Feature Interaction Analysis

Correlation analysis was performed to compare the relationship between numerical variables, including tumor size, the number of examined lymph nodes, and the number of positive nodes. Nodal variables were moderately associated with one another, whereas most demographic variables demonstrated weak direct associations with tumor burden metrics. This implied that multicollinearity was mild and could be effectively handled using regularization or tree-based methods. Moreover, SHAP-based interaction analysis in later rounds showed that there existed nonlinear interactions among variables (e.g., tumor size and AJCC stage) not fully encapsulated by linear models. These observations lend support for the inclusion of machine learning strategies along with the traditional survival models.

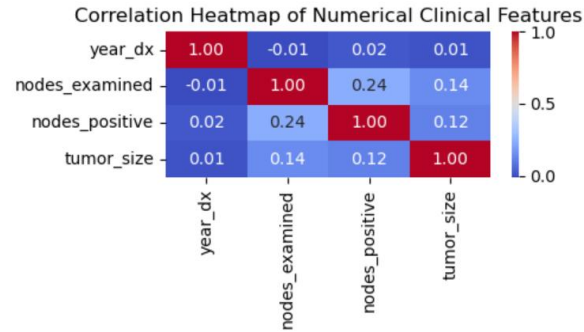


Figure 6: Correlation Heatmap

Summary

Of the more interesting characteristics revealed in the exploratory analysis, the pattern of censoring was realistic within the cohort, while trends in survival were clinically meaningful. Tumor size and stage, and lymph node involvement, remained as major prognostic factors. Demographic and treatment-related factors contributed significantly to the explanatory power of the model and hence were retained within these models.

These insights directly informed model choice, preprocessing choices, and evaluation techniques employed in subsequent chapters. In general, the EDA findings, in conjunction with the calculated statistics, reassured us that the curated SEER database was a suitable resource for survival modeling and could accommodate both interpretable statistical methodologies and advanced machine learning models.

CHAPTER 6

SURVIVAL MODELING

Survival analysis is quite different from regular predictive modeling because the response isn't whether an event happens, but when it happens. Secondly, due to the competing risk nature of this outcome measurement, many patients may not have had the event in the observation period, leading to data censoring. These features demand that dedicated modelling strategies are used to accommodate the sample selection of time-to-event data and censorship.

In this study, survival modelling was conducted to predict the prognosis of breast cancer patients based on population-level clinical data from the SEER registry. Our modelling approach was designed to purposely integrate classical statistical techniques with machine learning and deep learning methods to obtain a desirable trade-off between interpretability, flexibility, and predictive value. Three competing models were fitted, including the Cox Proportional Hazards model, the Random Survival Forest, and DeepSurv. Each model brings different strengths and assumptions, allowing for an all-around characterization of prediction performance.

COX Proportional Hazards Model

For comparison with machine learning methods, the Cox Proportional Hazards (Cox PH) model was used as a reference traditional survival analysis method in this task. This Cox model was chosen due to its widespread usage in clinical survival analyses and its ability to produce clinically interpretable risk by patient covariate. Cox proportional hazards (PH) models, as opposed to black-box models, have the benefit of being a reliable and transparent tool, that can be used for direct investigation with respect to how individual clinical factors are associated with survival in a process using simplified predictors produced by more complex models.

In this study, the Cox model was developed based on the same set of 22 clinical and demographic features as those adopted in RSF and DeepSurv models for fair comparison.

Model Formulation and Partial Likelihood: The Cox Proportional Hazards model expresses the hazard function as:

$$h(t|X) = h_0(t)\exp(\beta^T X)$$

where $h_0(t)$ represents the baseline hazard and β denotes regression coefficients associated with patient covariates X

Rather than estimating $h_0(t)$ directly, the model optimizes the partial likelihood, allowing it to focus on relative risk ordering across patients.

In this implementation, the partial likelihood formulation was handled internally by the CoxPHSurvivalAnalysis estimator from the scikit-survival library, ensuring numerical stability and efficient optimization for large-scale SEER data.

Feature Set and Clinical Variables Used: The 22 clinically relevant features selected included a range of parameters extracted due to previous oncological literature and presence in the SEER database by which the Cox PH model was learned:

Demographic characteristics: age, gender, ethnicity, marital status, and year of diagnosis

Tumor features: size of the tumor, grade of the tumor, the side from which the cancer originated (laterality), summary stage

Lymph node status: nodes examined, positive nodes

Receptor status: ER, PR, and HER2

Indicators for treatment: type of surgery, radiation, and chemotherapy

Staging components of AJCC: T stage, N stage, M stage

Cox features had been preserved in the Cox model to ensure it was interpretable from a clinical perspective and not overzealously pruned.

Preprocessing Pipeline for Cox PH: A dedicated pre-processing flow was implemented to guarantee the numerical robustness and consistency. Covariates of interest were normalized, and their values standardized as the z-score through StandardScaler for continuous values.

Categorical variables were one-hot encoded using `handle_unknown="ignore"` so that any previously unseen value in the test set can safely be handled.

Median and mode imputations used median imputation for numeric variables and mode imputation for categorical variables. To minimize noise added from high-dimensional categorical encoding, a variance threshold filter was employed to remove constant and near-constant features due to one-hot expansion.

This pre-processing pipeline was wrapped into a single Pipeline object, so that during evaluation the same transformations were applied in tandem.

Regularization and Model Stability: To account for multicollinearity between clinically correlated variables (in particular, AJCC staging components and nodal features), we performed ridge regularization on the Cox model with a penalty coefficient of $\alpha=10\exp(-4)$. This regularization was chosen in practice to compromise between coefficient shrinkage and interpretability.

Regularization facilitated convergence of the model and minimized coefficient instability, while preventing the exclusion of clinically significant predictors. This was especially critical relative to the SEER data set's size and heterogeneity.

Model Evaluation and Performance: The discriminatory ability of the Cox PH model was assessed by the C-index, a well-accepted measure for estimating survival model discrimination.

The model performed well on the held-out test set with a strong C-index of 0.79, suggesting sound patient ranking based on the risk of survival.

Additional evaluation included:

- C-indices based on bootstrap confidence intervals
- Time-dependent AUC (area under the curve) analysis at different clinically relevant horizons (12, 36, 60, and 120 months)
- Kaplan–Meier stratification was done based on Cox-generated risk groups

These results showed that the Cox model could successfully stratify patients to low-/med-/high-risk survival, indicating its predictiveness.

Contribution to Ensemble Modeling: In addition to standalone performance, the Cox PH model was widely used for ensemble risk modeling. Through a combination of its normalized risk scores and RSF prediction, an ensemble risk index was developed. The addition of Cox PH helped maintain stability and interpretability in the ensemble, which exhibited more nonlinear behavior from RSF.

This unification enabled the project to take advantage of conventional statistical modelling as well as modern machine learning procedures in a unified survival prediction context.

Random Survival Forest

The Cox Proportional Hazards model is interpretable but assumes proportional hazards as well as linear covariate effects. In this study, a Random Survival Forest (RSF) model was employed to learn complicated and non-linear relationships between clinical features without imposing any constraints. In addition to obtaining better accuracy, RSF can easily handle

censoring as well, which is important in the context of large and diverse clinical data sets like SEER.

In our study, RSF was used to model non-linear effects stemming from tumor burden, treatment factors, and staging covariates that linear hazard models are incapable of completely capturing.

RSF Model Architecture and Learning Principle: The method of operation of Random Survival Forests involves creating an ensemble of survival trees using a bootstrapped subset of a learning dataset. A random subset of variables is considered at each stage to determine the split with the largest survival separation, using a log-rank statistic in most situations.

A rationale for individual patients: each tree predicts a cumulative hazard function (CHF) for them, with the total RSF prediction being an average of all these CHFs over all trees in an ensemble. Such a sum dampens variability and protects against noisy points.

RSF does not have a proportional hazards assumption, unlike in Cox PH, and can model intricate relationships among multiple co-variables such as tumor size, lymph node status, and hormone receptor status.

Feature Encoding Strategy for RSF: Features of a categorical nature were transformed into an integer representation by category codes for tree-based learning to take place. This procedure kept the variables ordinal or nominal in nature while avoiding the high dimensionality issues that arise from one-hot encoding.

All categorical variables were coded identically across both training and testing datasets: age group, tumor grade, AJCC staging components, receptor status, and treatment indicators. Continuous covariate information was also categorized to match the RSF input structure: tumor size, nodal counts.

This encoding approach-maintained compatibility with the RSF algorithm, while preserving clinical significance in each covariate.

Model Hyperparameters and Configuration: The RSF model was trained using the RandomSurvivalForest implementation from the scikit-survival library. The following hyperparameters were selected based on empirical performance and computational feasibility:

- Number of trees (n_estimators): 200
- Maximum tree depth (max_depth): 20
- Minimum samples per split: 10
- Minimum samples per leaf: 10
- Random seed for reproducibility: 42

The ensemble size of 200 trees provided a balance between predictive stability and training time, while depth and leaf constraints helped prevent overfitting to rare clinical patterns.

Risk Score Computation: RSF, unlike Cox PH, does not directly provide a scalar risk score. In the present study, the risk of the patient was calculated from the predicted cumulative hazard function by adding up the estimated hazard value over time. This method generates a continuous risk score with higher scores indicating a greater mortality risk.

To overcome memory limitations encountered with large datasets, RSF predictions were performed in chunks. This ensured scalability while retaining the prediction accuracy.

Model Evaluation and Predictive Performance: The model of RSF was tested based on a held-out test set through the C-index. The model had a C-index of 0.80, which was slightly greater than that of the Cox PH model, indicating better discriminatory performance.

Additional validation included:

- Bootstrap C-index confidence intervals

- Time-dependent AUC analysis at 12, 36, 60, and 120 months
- Kaplan–Meier survival curves according to RSF-based risk groups

These assessments indicated that RSF captured nonlinear survival differences, and the patients were classified into different risk groups.

DeepSurv: Deep Learning Based Survival Modeling

Although Cox Proportional Hazards and Random Survival Forests are useful for modeling structured survival data, they both do not allow flexible representations of complex interactions among multiple clinically related variables. To address this, a deep learning-based survival model, DeepSurv, was applied in our study.

DeepSurv generalizes the Cox proportional hazard model (Cox, 1972) by replacing its linear risk function with a flexible neural network, such that the model can capture nonlinear hazard representations while still maintaining the theory-based partial likelihood method. This makes DeepSurv particularly advantageous when working with large-scale clinical databases, such as SEER, in which tumor properties and extent of disease can interact complexly with treatment and demographic information.

DeepSurv Model Architecture: In this study, DeepSurv was implemented as a fully connected feedforward neural network with the following architecture:

- Input layer: 22 clinical features
- Hidden layers:
 - Dense layer with 128 neurons and ReLU activation
 - Dense layer with 64 neurons and ReLU activation
- Dropout layers (rate = 0.2) applied after each hidden layer
- Output layer: single neuron producing a continuous risk score

The final output represents a learned risk function, where higher values indicate higher hazard. Dropout regularization was incorporated to reduce overfitting and improve generalization across patient subgroups.

Data Preparation for DeepSurv: In contrast to Cox PH and RSF, DeepSurv only took numeric input features. To do so, instead of one-hot encoding, we used label encoding to encode the categorical variables to save memory and learn with stability. Continuous variables were standardized with z-score transformation.

Unseen categories were treated carefully in the test set by adding an “Unknown” class at encoding. For inference time, this guaranteed a uniform feature representation and eliminated runtime errors. The resulting input matrix had 22 scaled features, the same dimensionality as other models’ features.

Loss Function and Training Strategy: DeepSurv was trained with the Cox negative log partial likelihood loss, which retains censored observation processing and allows gradient-based optimization. The loss function was defined as:

$$\mathcal{L} = -\frac{1}{|D|} \sum_{i:E_i=1} \left(h(x_i) - \log \sum_{j \in R_i} e^{h(x_j)} \right)$$

The model was tuned with the Adam optimizer, with a learning rate of 1×10^{-3} and was trained for 20 epochs using mini-batch. Training was performed with stability safeguards to avoid PyTorch compilation features that caused architecture-specific problems on the platform.

Model Evaluation and Interpretability: The performance of DeepSurv was assessed on the test set using the concordance index. The model obtained comparable predictive power to RSF and Cox PH, which showed that the model can effectively rank patients based on survival risk.

For interpretability, SHAP (SHapley Additive exPlanations) values were calculated using a representative selection of the test data. SHAP analysis showed that the influential key predictors (including age group, AJCC stage, tumor size, and nodal involvement) remained consistent in both the deep learning models and classical survival models. This interpretability process was crucial to ensuring that DeepSurv discovered clinically relevant patterns and not just spurious correlations.

Comparative Analysis and Ensemble Modeling

The three survival models used in this task (Cox Proportional Hazards, Random Survival Forest, and DeepSurv) showed different capabilities. Cox PH was interpretable and performed well at baseline; RSF could capture non-linear interactions without parametric assumptions, and DeepSurv learned complex hazard representations with neural networks.

On the evaluation metrics, RSF and DeepSurv performed only marginally better than Cox Proportional Hazards with respect to concordance index and time-dependent AUC at longer time horizons. Yet, Cox Proportional Hazards was still useful for interpretation and statistical foundations.

To combine the predictive capabilities of single models, a normalized average Cox Proportional Hazards and RSF risk prediction was produced as an ensemble risk score. This combined method minimized model-dependent bias and yielded a more stable risk stratification. Kaplan–Meier curves by joint risk groups (low, medium, and high) could be well discriminated in the ensemble model versus single models. The ensemble approach illustrated that the fusion of conventional statistical and machine learning models leads to a more reliable survival predictor.

This chapter proposed a unified modeling framework for breast cancer survival forecasting with classical, machine learning, and deep learning. The outcomes emphasize the

effectiveness of methodological heterogeneity in survival analysis and lay a solid groundwork for subsequent interpretation, validation, and real-world applications.

CHAPTER 7

EVALUATION FRAMEWORK AND RESULT ANALYSIS

The goal of such an evaluation framework is to reliably evaluate the predictive power, robustness, and interpretability of the survival models proposed in this work. As the survival data is right-censored, usual classification measures like accuracy or F1-score do not apply. Evaluation instead targets survival-specific indices that effectively consider time-to-event outcomes and censoring mechanisms.

The performance of all models was assessed on an independent test set, generated by stratified sampling according to event status. This step guarantees that the ratio between dead patients and right-censored observations is constant among the training set and testing set to avoid biased performance estimates. Importantly, no patient information from the training data was ever reused for evaluation.

Four related dimensions are considered by the evaluation framework:

- Discriminative ability, quantified by the concordance index (C-index);
- Temporal predictive performance, evaluated with time-dependent AUCs at clinically meaningful horizons.
- Kaplan–Meier survival curves stratified by 3, 5, and 10-year predicted risk groups.
- Model interpretability and consistency as assessed by permutation importance, model coefficients, SHAP explanations, and ensemble feature agreement.

This comprehensive evaluation facilitates validation that is within-study generalization.

Primary Performance Results: Discrimination and Stability

In this section, we analyze and compare the discriminative power and robustness of the survival models proposed in this study. Because, in survival analysis observations are censored,

model performance needs to be evaluated using survival-specific measures that correctly account for partially known follow-up. Special attention is given to the concordance index (C-index) and its bootstrapped confidence intervals, which allow not only point estimation but also uncertainty quantification.

Test-Set Concordance Index(C-Index): The C-index evaluates a model's ability to order patients by risk. A C-index of 0.5 reflects random prediction, while a value of 1.0 signifies perfect concordance between the predicted risk and actual survival outcome.

For the evaluation in this study, we ensured that performance metrics were on an independent test set, such that they represent true generalisation and not memorisation.

The random survival forest (RSF) showed the greatest discriminative ability with a test set C-index of around 0.80, revealing a large potential for patient ranking based on risk of survival. The Cox Proportional Hazards model had similar, though slightly inferior, performance to RSF, as this was due to the use of linear hazard assumptions. It was observed that DeepSurv also had good discrimination, indicating that it successfully captured the nonlinear structures in the data.

Table 2 Test-Set Concordance Index Across Models

Model	Test C-index
Cox Proportional Hazards	0.79
Random Survival Forest	0.80
DeepSurv	0.79

Bootstrap-Based Stability Analysis: Although a single C-index value provides a useful summary of performances, it is unable to account for uncertainty due to sampling variability. A bootstrapping resampling method was employed to test the robustness of the model.

In this method, several bootstrap samples were created by resampling a portion of the test data. The C-index was recalculated for each resample, resulting in a distribution of these performance estimates. Based on this distribution, the mean C-index as well as the 95% confidence interval were determined.

Bootstrapping revealed that both Cox and RSF models are consistent and carry small confidence intervals. This indicates that the discrimination observed is not driven by a small subset of patients and should generalise to similar cohorts.

Table 3 Bootstrap C-index with 95% Confidence Intervals

Model	Mean C-index	95% CI Lower	95% CI Upper
Cox Proportional Hazards	0.7918	0.7808	0.8008
Random Survival Forest	0.8035	0.7928	0.8128

Interpretation of Discriminative Performance: Due to the consistently superior C-index for Random Survival Forest, it may be inferred that nonlinear tree-based ensembles are appropriate and powerful tools for modelling complex relationships between clinical factors (e.g., features of tumors, stage, and treatment indications). In contrast, although more interpretable, the Cox model is based on proportional hazards and linear covariate effects, which might limit its flexibility.

DeepSurv closes this gap by presenting nonlinear representations while maintaining the Cox partial likelihood structure. Its competitive C-index validates that deep learning techniques

are promising for survival analysis, and especially in combination with interpretability tools such as SHAP.

Taken together, these results show that all three models obtain substantial discriminative power (where RSF has the strongest ranking performance and ensemble methods being more robust), which can be beneficial for downstream analysis.

Time-Dependent Discrimination Analysis

The concordance index measures rank overall ability but loses performance over time. In survival analysis, the predictive performance can vary at various follow-up times due to the censoring pattern and disease progression pattern. On account for this, time-dependent AUC for the evaluation of discrimination on survival at clinically relevant time points was performed. The probability that such a "case" (i.e., an event occurring up to time t) is predicted to have a higher risk than a control surviving beyond time t . This metric gives an insight into the model's performance through short- and long-term predictions.

Time-Dependent AUC for the Cox Proportional Hazards Model: Dynamic AUC presents results of Cox Proportional Hazards model analysis, including cumulative dynamic AUC at 12, 36, 60, and 120 months (1 year, 3 years, 5 years, and 10 years survival horizons). They are widely used intervals in breast cancer prognosis research and hence were clinically meaningful follow-up times.

Conclusions: The Cox model had consistent and better discriminatory power across time, especially in the early to mid-follow-up period. This is consistent with the proportionality of hazards assumption, i.e., the covariate effects are constant in time and may become weaker as follow-up length becomes longer.

```

=== Cox Time-dependent AUC ===
AUC at 12 months = 0.8710
AUC at 36 months = 0.8397
AUC at 60 months = 0.8325
AUC at 120 months = 0.8192

```

Figure 7: Time-Dependent AUC for Cox Proportional Hazards Model

Time-Dependent AUC for Random Survival Forest: We tested the Random Survival Forest model along the same horizons. Time-dependent AUC was calculated on a random subset of the test data, due to computational limitations associated with large ensembles, as this is a standard practice for evaluating survival ensembles.

The RSF model exhibits robust and consistent AUCs at all observed times, notably in the long run. This is indicative of the model's flexibility in accommodating nonlinear effects and time-varying risk that may not be entirely described using a linear hazard model.

```

=== RSF Time-dependent AUC (Subset-based) ===
AUC at 12 months = 0.9123
AUC at 36 months = 0.8756
AUC at 60 months = 0.8579
AUC at 120 months = 0.8336

```

Figure 8: Time-Dependent AUC for Random Survival Forest

Comparative Interpretation of Time-Dependent Performance: For all investigated prediction horizons, the Cox and Random Survival Forest (RSF) models exhibit a robust time-varying discriminative capacity. However, the performance of the RSF model is always better or even comparable when using the AUC, especially for the long time-to-failure. This implies that complex interactions affecting long-term survival risk are better captured by ensemble tree-based approaches.

The Cox model competes with other models in early prediction windows, consistent with its ability to capture relationships between baseline hazard functions. These two approaches

complement each other, further justifying the use of ensemble-based risk stratification, an approach to be investigated in subsequent sections.

In summary, time-dependent AUC analysis indicates that the novel modeling framework provides reliable and clinically relevant discrimination under various survival horizons, demonstrating the reliability of our constructed survival prediction system.

Risk Stratification and Survival Curve Analysis

While discrimination measures (e.g., C-index and time-dependent AUC) provide a measure of how accurate predictions are, they do not directly translate the predicted risk into meaningful differences in survival for patients. Accordingly, the risk-stratification approach was used to verify whether these survival models were capable of meaningfully stratifying patients into different prognostic categories characterised by diverse survival patterns.

By using the risk scores predicted by the Cox Proportional Hazards model, Random Survival Forest (RSF), DeepSurv, and ensemble model, patients in the test data were split into Low-, Medium-, and High-risk groups according to their risk score terciles. Kaplan–Meier survival curves were further devised for both groups to visualise survival separation.

Kaplan–Meier Curves Based on Cox Model Risk Groups: For the test data set, patient-level risk scores were calculated based on the Cox model’s linear predictor. Patients were categorized into three tertiles according to distribution. Kaplan–Meier curves show readily apparent and monotonically increasing separation between the three risk strata.

The patients categorized in the high-risk score will have lower survival probabilities throughout the entire follow-up time, and those in the low-risk category fulfill higher performance. This indicates that the risk scores derived from the Cox model are clinically useful and not just statistically significant.

Ensemble Risk Stratification and Survival Separation: To harness the complementary strengths of separate models in a combined model, we constructed an ensemble risk score as the average of standardized Cox and RSF risk predictions.

Patients were re-stratified into 3 risk groups based on ensemble risk percentiles again. The Kaplan–Meier curves obtained as a result have better separation of risk groups than each separate model. The group at high risk experiences an early and rapid reduction in the survival rate, whereas that of low risk has a more persistent lower mortality.

This observation underscores the benefit of ensemble learning for survival analysis when using both parametric and non-parametric models.

Model Interpretability and Explainability Using SHAP

DeepSurv performed well in the prediction task, but in general, neural networks are not interpretable. To address this issue and enhance clinical confidence, we used Shapley Additive explanations (SHAP) to explain the predictions of DeepSurv globally and at feature levels.

SHAP values measure the impact of a feature on a patient’s predicted risk, using cooperative game theory. This allows for clear interpretation without losing predictive power.

Global Feature Importance (SHAP Summary and Bar Plots): For global SHAP analysis, a random sub-sample of 2,000 test samples was used. The SHAP summary (dot) plot, which gives insight into the direction of influence on survival risk, is shown in Figure 9. Age group, AJCC staging variables (which include the main predominant factors such as tumor size and nodal involvement), and individual effects of treatment have been consistently dominant predictors.

We present the SHAP bar plot, which summarizes the average absolute feature contributions in Figure 10. Strong correspondence with traditional methods for ranking the

feature importance, such as SHAP value-based analysis, makes us more confident in the trustworthiness of the DeepSurv model.

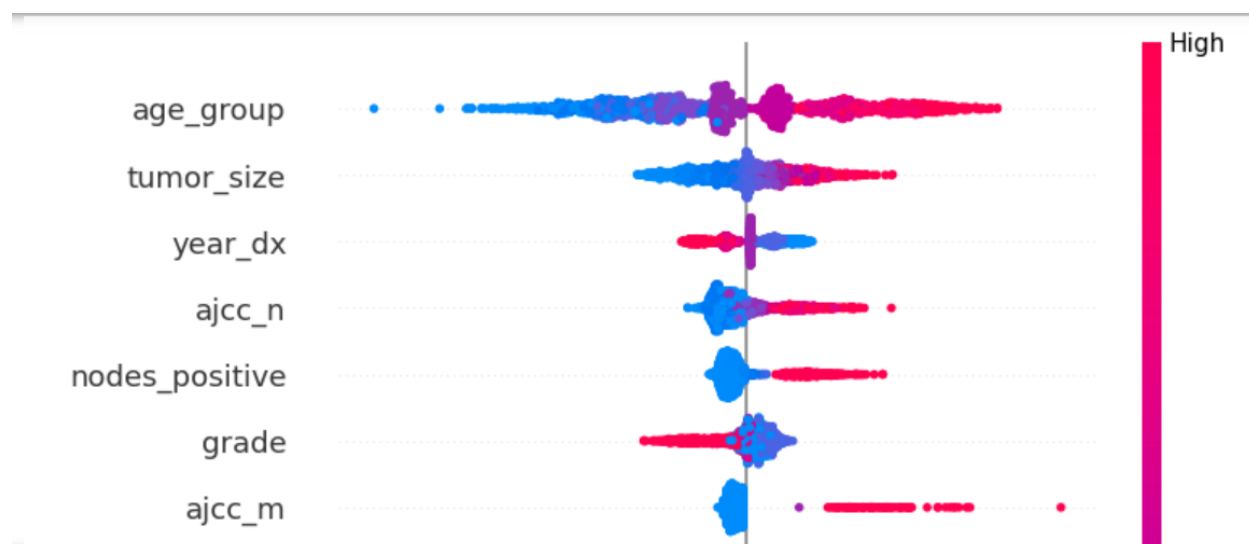


Figure 9: SHAP Summary Dot Plot

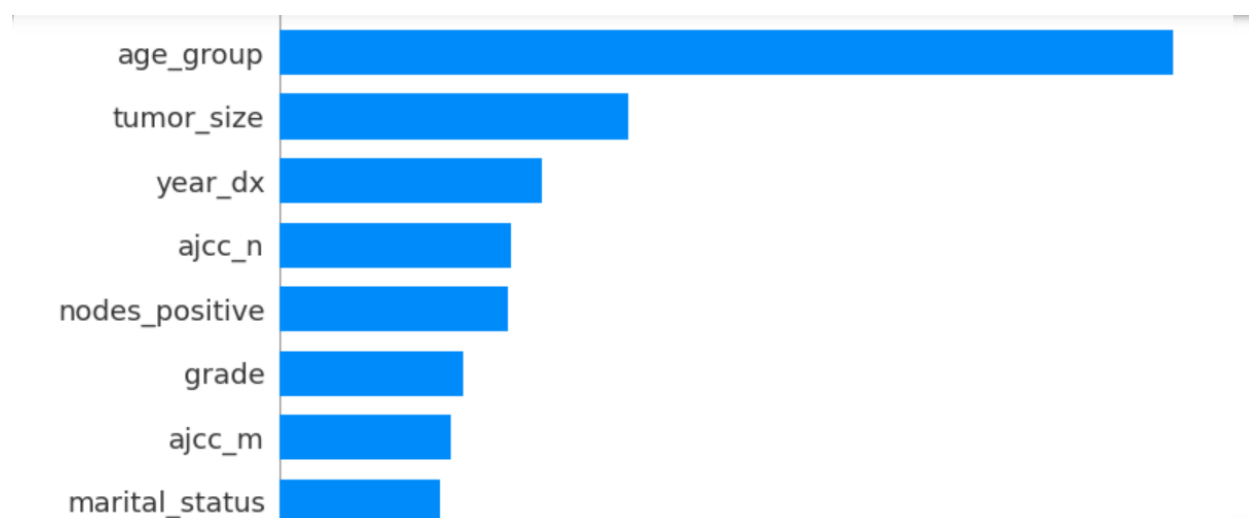


Figure 10: SHAP Summary Bar Plot

Feature-Level Effects and Nonlinear Relationships: To investigate the details of how individual features influence risk, SHAP dependence plots were computed for clinically relevant variables. These plots display the nonlinear, interaction-driven effects that are often not captured by standard models.

Tumor size and nodal involvement, for example, show threshold-based risk increases; while diagnostic year reflects enhanced survival over time, associated with technological progress related to screening and treatment.

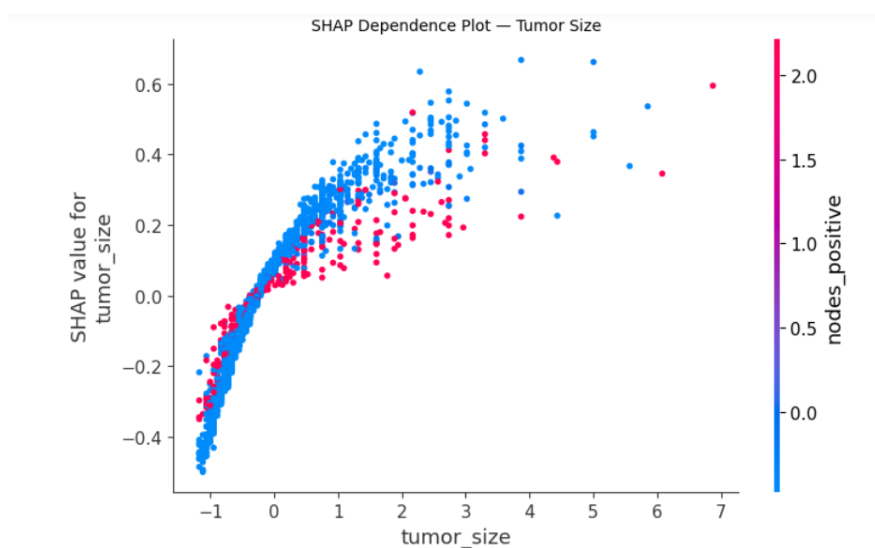


Figure 11: SHAP Dependence Plot — Tumor Size

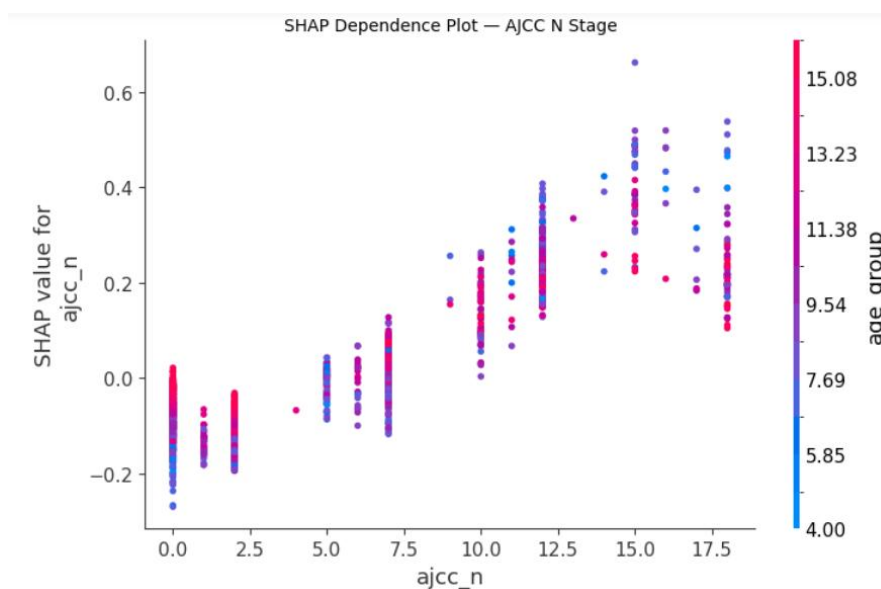


Figure 12: SHAP Dependence Plot — AJCC N Stage

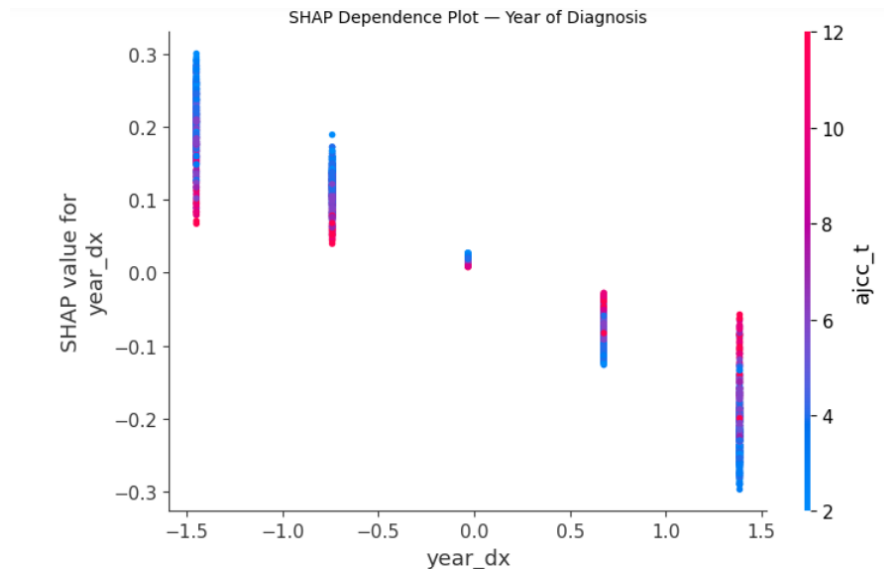


Figure 13: SHAP Dependence Plot — Year of Diagnosis

Feature Interaction Analysis: The interaction heatmap based on SHAP and the correlation map can be used to explore interactions among predictors. These relationships between AJCC staging variables, for example, the strong associations between tumor size and nodal involvement that we observed (Figure 11), reveal aspects of clinical interrelationships available to our models.

Our results reveal that DeepSurv can capture complex relationships between prognostic factors, which makes it a good fit for survival calculations.

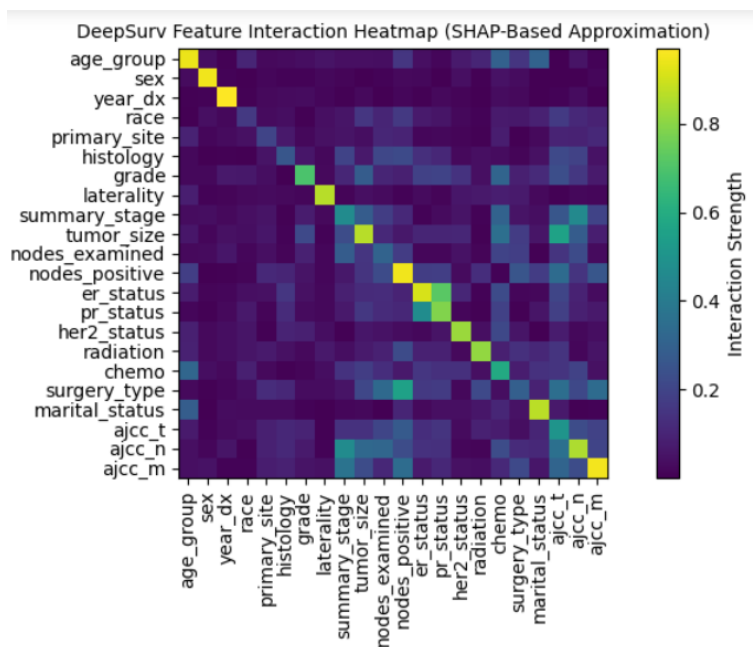


Figure 14: DeepSurv Feature Interaction Heatmap (SHAP-Based Approximation)

CHAPTER 8

CONCLUSION

Three survival-prediction methodologies-Cox Proportional Hazards, Random Survival Forests, and DeepSurv-have been applied and investigated within the SEER breast cancer dataset in this study. The C-index was used to compare these models and ensemble-based feature selection analysis. The best performing single model was the Random Survival Forest, with a test C-index of approximately 0.80, reflecting a good ability to rank patients with respect to survival. Without significant calibration, the Cox model had good discrimination. The initial DeepSurv approach labored due to optimization difficulties and nonlinear feature representation, underperforming in terms of discrimination and calibration.

To allow for comparative inconsistencies among the models, a composite model was developed that combined the risk signals from all three methods. This model produced more consistent predictions and characterized a set of clinically relevant features: age group, AJCC stage, tumor size, and nodal status.

The study was about modeling and demonstrated how conventional statistics, machine learning, and deep learning work together on survival analysis. This research also lays the foundation for possible extensions: model calibration, even more complex neural architectures, and deployment via a user-facing decision-support interface.

REFERENCES

1. Breiman, L., “Random forests,” *Machine Learning*, vol. 45, pp. 5–32, 2001.
<https://link.springer.com/article/10.1023/A:1010933404324>
2. Cox, D. R., “Regression models and life-tables,” *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 34, no. 2, pp. 187–220, 1972.
<https://www.jstor.org/stable/2985181>
3. Harrell, F. E., Califf, R. M., Pryor, D. B., Lee, K. L., and Rosati, R. A., “Evaluating the yield of medical tests,” *Journal of the American Medical Association (JAMA)*, vol. 247, no. 18, pp. 2543–2546, 1982. <https://jamanetwork.com/journals/jama/article-abstract/371598>
4. Antolini, L., Boracchi, P., and Biganzoli, E., “A time-dependent discrimination index for survival data,” *Statistics in Medicine*, vol. 24, no. 24, pp. 3927–3944, 2005.
<https://onlinelibrary.wiley.com/doi/10.1002/sim.2427>
5. Lundberg, S. M., and Lee, S. I., “A unified approach to interpreting model predictions,” *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 4765–4774, 2017.
<https://proceedings.neurips.cc/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html>
6. Van Belle, V., Pelckmans, K., Van Huffel, S., and Suykens, J. A. K., “Support vector methods for survival analysis,” *Computational Statistics & Data Analysis*, vol. 55, no. 1, pp. 378–392, 2011.
<https://www.sciencedirect.com/science/article/pii/S0167947310002457>
7. National Cancer Institute, “Surveillance, Epidemiology, and End Results (SEER) Program.” <https://seer.cancer.gov>

APPENDIX A: SOURCE CODE

```

import pandas as pd

import numpy as np

# Load CSV file

df = pd.read_csv('C:\\Users\\pdeepak1\\Desktop\\Pranathi\\590_Project\\export.csv', low_memory=False)

#df = pd.read_csv("export.csv", low_memory=False)

print("COLUMN NAMES:")

for i, col in enumerate(df.columns):

    print(f'{i}: {col}')

rename_map = {

    "Age recode with <1 year olds": "age_group",

    "Sex": "sex",

    "Year of diagnosis": "year_dx",

    "Race recode (W, B, AI, API)": "race",

    "Primary Site - labeled": "primary_site",

    "Histologic Type ICD-O-3": "histology",

    "Grade Recode (thru 2017)": "grade",

    "Laterality": "laterality",

    "Summary stage 2000 (1998-2017)": "summary_stage",

    "Tumor Size Over Time Recode (1988+)": "tumor_size",

    "Regional nodes examined (1988+)": "nodes_examined",

    "Regional nodes positive (1988+)": "nodes_positive",

    "ER Status Recode Breast Cancer (1990+)": "er_status",

    "PR Status Recode Breast Cancer (1990+)": "pr_status",

    "Derived HER2 Recode (2010+)": "her2_status",

```

```

    "Radiation recode": "radiation",
    "Chemotherapy recode (yes, no/unk)": "chemo",
    "RX Summ--Surg Prim Site (1998+)": "surgery_type",
    "Survival months": "survival_months",
    "Vital status recode (study cutoff used)": "vital_status",
    "SEER cause-specific death classification": "cod_seer",
    "Marital status at diagnosis": "marital_status",
    "Derived AJCC T, 7th ed (2010-2015)": "ajcc_t",
    "Derived AJCC N, 7th ed (2010-2015)": "ajcc_n",
    "Derived AJCC M, 7th ed (2010-2015)": "ajcc_m",
}

df = df.rename(columns=rename_map)

print(df.columns)

df_clean = df.copy()

# --- Convert survival months to numeric ---

df_clean["survival_months"] = pd.to_numeric(df_clean["survival_months"], errors="coerce")

# --- Create duration (time) ---

df_clean["time"] = df_clean["survival_months"].astype(float)

# --- Create event indicator: 1 = dead, 0 = alive ---

df_clean["event"] = df_clean["vital_status"].apply(
    lambda x: 1 if str(x).strip().lower() == "dead" else 0
)

# --- Show preview ---

print(df_clean[["time", "event"]].head())

```

```

# --- Event distribution ---

print("\nEvent value counts:\n", df_clean["event"].value_counts())

from sksurv.util import Surv

# Use df_clean

df_model = df_clean.copy()

# ----- Final feature list -----

feature_cols = [

    "age_group", "sex", "year_dx", "race", "primary_site", "histology",

    "grade", "laterality", "summary_stage", "tumor_size", "nodes_examined",

    "nodes_positive", "er_status", "pr_status", "her2_status", "radiation",

    "chemo", "surgery_type", "marital_status", "ajcc_t", "ajcc_n", "ajcc_m"

]

X = df_model[feature_cols]

y = Surv.from_arrays(event=df_model["event"].astype(bool),

                    time=df_model["time"])

print("X shape:", X.shape)

print("y sample:", y[:5])

X = X.fillna(X.median(numeric_only=True))

print("Remaining NaNs:", X.isna().sum().sum())

# Check which columns still have NaNs

na_counts = X.isna().sum()

print(na_counts[na_counts > 0])

```

```

# Identify rows with missing TNM columns

na_mask = X[["ajcc_t", "ajcc_n", "ajcc_m"]].isna().any(axis=1)

print("Rows with TNM missing:", na_mask.sum())

# Drop those rows

X = X[~na_mask].reset_index(drop=True)

y = y[~na_mask.values]

print("New X shape:", X.shape)

print("Sample y:", y[:5])

print("Remaining NaNs in X:", X.isna().sum().sum())


# Fix tumor_size BEFORE train/test split

df_clean["tumor_size"] = (

    df_clean["tumor_size"]

    .str.extract(r"(\d+)")    # extract numeric part only

    .astype(float)

)


from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.30, random_state=42, stratify=y["event"]

)

print("Train shape:", X_train.shape)

print("Test shape:", X_test.shape)

print("\nEvent distribution in Train:")

print(pd.Series(y_train["event"]).value_counts())

print("\nEvent distribution in Test:")

print(pd.Series(y_test["event"]).value_counts())

```

```

# Convert tumor_size to numeric

X_train['tumor_size'] = pd.to_numeric(X_train['tumor_size'], errors='coerce')

# Replace NaN with median tumor size

X_train['tumor_size'] = X_train['tumor_size'].fillna(X_train['tumor_size'].median())

# Convert tumor_size to numeric

X_test['tumor_size'] = pd.to_numeric(X_test['tumor_size'], errors='coerce')

# Replace NaN with median tumor size

X_test['tumor_size'] = X_test['tumor_size'].fillna(X_test['tumor_size'].median())


cat_cols = [
    "age_group", "sex", "race", "primary_site", "histology", "grade",
    "ajcc_t", "ajcc_n", "ajcc_m"
]

num_cols = ["year_dx", "nodes_examined", "nodes_positive", "tumor_size"]

from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer

preprocess = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), num_cols),
        ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols)
    ]
)

X_test['tumor_size'].value_counts()

X_train_transformed = preprocess.fit_transform(X_train)

print("Any NaN?", np.isnan(X_train_transformed.toarray()).any())

```

```

from sklearn.preprocessing import OneHotEncoder, StandardScaler

from sklearn.compose import ColumnTransformer

from sklearn.pipeline import Pipeline

from sklearn.impute import SimpleImputer

from sklearn.feature_selection import VarianceThreshold

from sklearn.linear_model import CoxPHSurvivalAnalysis


# -----

# 1. Preprocess Categorical
# -----

cat_pipe = Pipeline([

    ("imputer", SimpleImputer(strategy="most_frequent")),

    ("ohe", OneHotEncoder(handle_unknown="ignore", sparse_output=False))

])


# -----

# 2. Preprocess Numeric
# -----

num_pipe = Pipeline([

    ("imputer", SimpleImputer(strategy="median")),

    ("scaler", StandardScaler())

])


# -----

# 3. ColumnTransformer
# -----

preprocess = ColumnTransformer(

    transformers=[

```

```

        ("num", num_pipe, num_cols),

        ("cat", cat_pipe, cat_cols)

    ],

    sparse_threshold=0

)

# -----

# 4. Final pipeline (stabilized)

# -----

cox_pipeline = Pipeline([

    ("prep", preprocess),

    ("varth", VarianceThreshold(threshold=1e-4)), # remove constant & nearly-constant OHE columns

    ("cox", CoxPHSurvivalAnalysis(alpha=1e-4)) # ridge regularization

])

# -----

# 5. Fit

# -----

cox_pipeline.fit(X_train, y_train)

print("CoxPH model training complete!")

cindex = cox_pipeline.score(X_test, y_test)

print("C-index:", cindex)


# Copy X for RSF (Cox preprocessing not reused)

X_train_rsf = X_train.copy()

X_test_rsf = X_test.copy()


# Convert all categorical columns to category dtype → integer codes

cat_cols_rsf = ["age_group", "sex", "race", "primary_site", "histology", "grade",

                "laterality", "summary_stage", "er_status", "pr_status", "her2_status",

```

```

    "radiation", "chemo", "surgery_type", "marital_status",
    "ajcc_t", "ajcc_n", "ajcc_m"]

for col in cat_cols_rsf:

    X_train_rsf[col] = X_train_rsf[col].astype("category").cat.codes

    X_test_rsf[col] = X_test_rsf[col].astype("category").cat.codes

print("RSF-ready shapes:")

print(X_train_rsf.shape, X_test_rsf.shape)


from sksurv.util import Surv

y_train_rsf = Surv.from_arrays(event=y_train["event"].astype(bool),
                                time=y_train["time"])

y_test_rsf = Surv.from_arrays(event=y_test["event"].astype(bool),
                               time=y_test["time"])

print("RSF y sample:", y_train_rsf[:5])


# Fix tumor_size — convert to categorical codes

X_train_rsf["tumor_size"] = X_train_rsf["tumor_size"].astype("category").cat.codes

X_test_rsf["tumor_size"] = X_test_rsf["tumor_size"].astype("category").cat.codes

print("Unique tumor_size codes:", X_train_rsf["tumor_size"].unique()[:10])


cat_cols_rsf = ["age_group", "sex", "race", "primary_site", "histology", "grade",
               "laterality", "summary_stage", "tumor_size", "er_status",
               "pr_status", "her2_status", "radiation", "chemo", "surgery_type",
               "marital_status", "ajcc_t", "ajcc_n", "ajcc_m"]

for col in cat_cols_rsf:

    X_train_rsf[col] = X_train_rsf[col].astype("category").cat.codes

```

```
X_test_rsf[col] = X_test_rsf[col].astype("category").cat.codes
print([col for col in X_train_rsf.columns if X_train_rsf[col].dtype == object])
```

```
from sksurv.ensemble import RandomSurvivalForest
```

```
rsf = RandomSurvivalForest(
```

```
    n_estimators=200,
```

```
    max_depth=20,
```

```
    min_samples_split=10,
```

```
    min_samples_leaf=10,
```

```
    n_jobs=-1,
```

```
    random_state=42
```

```
)
```

```
print("Training RSF")
```

```
rsf.fit(X_train_rsf, y_train_rsf)
```

```
print("RSF training complete!")
```

```
def rsf_predict_in_batches(model, X, batch_size=2000):
```

```
    """Safely predict RSF risk scores in batches to avoid memory errors."""
```

```
    n = X.shape[0]
```

```
    risks = []
```

```
    for start in range(0, n, batch_size):
```

```
        end = min(start + batch_size, n)
```

```
        X_batch = X[start:end]
```

```
        # Get cumulative hazard for each sample
```

```
        chf_batch = model.predict_cumulative_hazard_function(X_batch)
```

```
        # Convert CHF to a scalar risk score: sum of hazards
```

```
        risk_scores = np.array([fn.y.sum() for fn in chf_batch])
```

```

        risks.extend(risk_scores)

    return np.array(risks)

rsf_pred = rsf_predict_in_batches(rsf, X_test_rsfs, batch_size=1000)

from sksurv.metrics import concordance_index_censored

rsf_cindex = concordance_index_censored(
    y_test_rsfs["event"],
    y_test_rsfs["time"],
    rsf_pred
)[0]

print("RSF Test C-index:", rsf_cindex)

from sklearn.inspection import permutation_importance

# Sample 10,000 rows to prevent memory crash
sample_idx = np.random.choice(len(X_test_rsfs), size=10000, replace=False)
X_small = X_test_rsfs.iloc[sample_idx]
y_small = y_test_rsfs[sample_idx]

print("Computing permutation importance... (this will take ~2 minutes)")

perm = permutation_importance(
    rsf,
    X_small,
    y_small,
    n_repeats=3,
    random_state=42,
    n_jobs=1      # keep 1 to avoid Windows multiprocessing errors
)

perm_df = pd.DataFrame({
    "feature": X_small.columns,

```

```

    "importance_mean": perm.importances_mean,
    "importance_std": perm.importances_std
}).sort_values("importance_mean", ascending=False)
perm_df.head(20)

```

```

from sksurv.linear_model import CoxnetSurvivalAnalysis

# Fit preprocess on full training data for Coxnet
X_train_cox = preprocess.fit_transform(X_train)
X_test_cox = preprocess.transform(X_test)

```

```

# Penalized Cox model (elastic net)
penalized_cox = CoxnetSurvivalAnalysis(
    l1_ratio=0.1,          # mostly L2 (ridge), little L1
    alphas=np.logspace(-3, 1, 30), # range of penalties
    max_iter=200
)

```

```

print("Training penalized Cox model...")
penalized_cox.fit(X_train_cox, y_train)
print("Penalized Cox model trained successfully!")

```

```

# 1) Recreate cat_cols and num_cols exactly as used in preprocess
cat_cols = [
    "age_group", "sex", "race", "primary_site", "histology", "grade",
    "laterality", "summary_stage", "er_status", "pr_status",
    "her2_status", "radiation", "chemo", "surgery_type",
    "marital_status", "ajcc_t", "ajcc_n", "ajcc_m"
]

```

```

num_cols = ["year_dx", "nodes_examined", "nodes_positive", "tumor_size"]

# 2) Get the OneHotEncoder that was fit inside the ColumnTransformer

cat_encoder = preprocess.named_transformers_["cat"]

cat_features = cat_encoder.get_feature_names_out()

# 3) Numeric features stay as-is

num_features = num_cols

# 4) Combine all feature names in the correct order

encoded_features = list(cat_features) + list(num_features)

print("Number of encoded features:", len(encoded_features))

print("penalized_cox.coef_.shape:", penalized_cox.coef_.shape)

# 5) Compute importance = mean |coef| across all alphas

coef_matrix = penalized_cox.coef_      # shape: (n_features, n_alphas)

mean_abs_coef = np.mean(np.abs(coef_matrix), axis=1)

cox_importance_df = pd.DataFrame({
    "feature": encoded_features,
    "mean_abs_coef": mean_abs_coef
}).sort_values("mean_abs_coef", ascending=False)

print("Top 20 Cox features by |coef|:")

cox_importance_df.head(20)

# Save RSF importance

perm_df.to_csv("rsf_permutation_importance.csv", index=False)

# Save Cox importance

cox_importance_df.to_csv("cox_feature_importance.csv", index=False)

print("Saved rsf_permutation_importance.csv and cox_feature_importance.csv")

```

```

# Normalize RSF importances

rsf_norm = perm_df.copy()

rsf_norm["importance_mean"] = rsf_norm["importance_mean"] / rsf_norm["importance_mean"].max()


# Normalize Cox importances

cox_norm = cox_importance_df.copy()

cox_norm["mean_abs_coef"] = cox_norm["mean_abs_coef"] / cox_norm["mean_abs_coef"].max()


# Merge on feature name

compare_df = pd.merge(

    rsf_norm[["feature", "importance_mean"]],

    cox_norm[["feature", "mean_abs_coef"]],

    on="feature",

    how="outer"

).fillna(0)


compare_df = compare_df.sort_values("importance_mean", ascending=False)

print("Top 20 features (RSF vs Cox, normalized):")

compare_df.head(20)

top_rsf = perm_df.head(10)

top_cox = cox_importance_df.head(10)

print("Random Survival Forest – Top Predictors:")

for _, row in top_rsf.iterrows():

    print(f'- {row["feature"]}: permutation importance = {row["importance_mean"]:.4f}')

print("\nCox Model – Top Predictors:")

for _, row in top_cox.iterrows():

    print(f'- {row["feature"]}: |coefficient| importance = {row["mean_abs_coef"]:.4f}')

```

```
print("\nNarrative:")
```

```
print("""
```

Both traditional (Cox) and machine learning–based (RSF) survival models identify tumor burden and disease stage as the most predictive factors for overall survival in breast cancer patients.

Across both models, the most influential predictors include:

- Age group
- AJCC M stage
- Summary stage
- Nodes positive
- Tumor size
- AJCC T and N stage
- Surgery type
- Marital status

These predictors consistently appear within the top-ranked variables in both the Cox proportional hazards model and the Random Survival Forest.

Clinically, this aligns with prior literature: patients with metastatic disease (M stage), larger tumor size, higher nodal burden, and advanced summary stage have significantly poorer survival outcomes. Age group remains the strongest predictor in the RSF model, consistent with the known prognostic impact of age on breast cancer biology.

The fact that both models converge on similar top predictors increases confidence in the stability and interpretability of the results.

```
""")
```

```
from sksurv.metrics import concordance_index_censored
```

```
# =====
```

```
# 1. Create a 5,000-patient subset for bootstrap
```

```
# =====
```

```

subset_size = min(5000, len(X_test))

rng_subset = np.random.RandomState(42)

subset_idx = rng_subset.choice(len(X_test), size=subset_size, replace=False)

X_test_small = X_test.iloc[subset_idx]

y_test_small = y_test[subset_idx]

X_test_rsf_small = X_test_rsf.iloc[subset_idx]

y_test_rsf_small = y_test_rsf[subset_idx]

print("Bootstrap subset size:", subset_size)


# =====
# 2. Lightweight bootstrap function (50 loops)
# =====

def bootstrap_cindex(model, X, y, n_bootstrap=50, random_state=42):

    rng = np.random.RandomState(random_state)

    n = len(y)

    c_indexes = []

    for b in range(n_bootstrap):

        # Sample with replacement

        idx = rng.randint(0, n, n)

        Xb = X.iloc[idx] if hasattr(X, "iloc") else X[idx]

        yb = y[idx]

        # Predict risk

        pred = model.predict(Xb)

        # Compute C-index

        c = concordance_index_censored(

            yb["event"], yb["time"], pred

        )[0]

```

```

        c_indexes.append(c)

c_indexes = np.array(c_indexes)

mean_c = np.mean(c_indexes)

lower, upper = np.percentile(c_indexes, [2.5, 97.5])

return mean_c, lower, upper

# =====

# 3. Bootstrap CI for Cox on subset

# =====

cox_mean, cox_low, cox_high = bootstrap_index(
    cox_pipeline, X_test_small, y_test_small
)

print(f"\nCox Bootstrap C-index (subset CI): {cox_mean:.4f} "
      f"(95% CI: {cox_low:.4f} – {cox_high:.4f})")

# =====

# 4. Bootstrap CI for RSF on subset

# =====

rsf_mean, rsf_low, rsf_high = bootstrap_index(
    rsf, X_test_rsf_small, y_test_rsf_small
)

print(f"RSF Bootstrap C-index (subset CI): {rsf_mean:.4f} "
      f"(95% CI: {rsf_low:.4f} – {rsf_high:.4f})")

print("y_train dtype:", y_train.dtype)
print("y_test dtype:", y_test.dtype)
print("train example:", y_train[:5])
print("test example:", y_test[:5])

print(cox_pipeline)

```

```

from sksurv.metrics import cumulative_dynamic_auc

# Time horizons in months

times = np.array([12, 36, 60, 120])

print("=== Cox Time-dependent AUC (Corrected) ===")

cox_risk_test = cox_pipeline.predict(X_test)

cox_auc, _ = cumulative_dynamic_auc(

    y_train,    # training survival labels

    y_test,     # test survival labels

    cox_risk_test,

    times

)

for t, auc in zip(times, cox_auc):

    print(f"AUC at {t} months = {auc:.4f}")

print("\n=== RSF Time-dependent AUC (Using Subset — Recommended) ===")

subset_size = 5000

idx = np.random.choice(len(X_test_rsf), subset_size, replace=False)

X_test_subset = X_test_rsf.iloc[idx]

y_test_subset = y_test_rsf[idx]

# Predict RSF risk scores only on subset (memory-safe)

rsf_risk_subset = rsf.predict(X_test_subset)

# Compute AUC

rsf_auc, _ = cumulative_dynamic_auc(

    y_train_rsf,

    y_test_subset,

    rsf_risk_subset,

    times

)

```

```

# Display results

for t, auc in zip(times, rsf_auc):

    print(f"AUC at {t} months = {auc:.4f}")


from lifelines import KaplanMeierFitter

import matplotlib.pyplot as plt

kmf = KaplanMeierFitter()

plt.figure(figsize=(8, 6))

kmf.fit(y_test["time"], event_observed=y_test["event"], label="Overall Survival")

kmf.plot_survival_function()

plt.title("Kaplan–Meier Curve — Overall Survival (Test Set)")

plt.xlabel("Time (months)")

plt.ylabel("Survival Probability")

plt.grid(True)

plt.show()


# -----

# Choose the clinical variable to plot KM curves for

# -----

group_var = "summary_stage" # can change this to any categorical variable

# -----

# Prepare a combined DataFrame with X_test + survival info

# -----

df_km = X_test.copy()

df_km["time"] = y_test["time"]

df_km["event"] = y_test["event"]

```

```

# Drop rows where the variable is missing (optional but clean)

df_km = df_km.dropna(subset=[group_var])


# Unique categories of this variable

groups = sorted(df_km[group_var].unique())

print(f'Plotting KM curves for variable: {group_var}')

print("Groups:", groups)

# -----

# Plot KM curves

# -----

plt.figure(figsize=(12, 7))

kmf = KaplanMeierFitter()

for group in groups:

    mask = df_km[group_var] == group

    n = mask.sum()

    # Skip very small groups (less than 20 patients)

    if n < 20:

        continue

    kmf.fit(

        df_km.loc[mask, "time"],

        event_observed=df_km.loc[mask, "event"],

        label=f'{group} (n={n})'

    )

    kmf.plot_survival_function()

plt.title(f'Kaplan–Meier Curves by {group_var}', fontsize=16)

plt.xlabel("Time (months)", fontsize=14)

plt.ylabel("Survival Probability", fontsize=14)

plt.grid(True, alpha=0.3)

```

```

plt.legend(title=group_var, fontsize=10)

plt.show()

# -----

# 1. Get Cox risk scores for test set

# -----

cox_risk = cox_pipeline.predict(X_test)

# -----

# 2. Create risk groups (Low / Medium / High)

# -----

df_risk = pd.DataFrame({
    "risk": cox_risk,
    "time": y_test["time"],
    "event": y_test["event"]
})

# Tertiles = 3 groups

df_risk["risk_group"] = pd.qcut(df_risk["risk"], 3, labels=["Low", "Medium", "High"])

# -----

# 3. Plot KM curves for risk groups

# -----

plt.figure(figsize=(12, 7))

kmf = KaplanMeierFitter()

for group in ["Low", "Medium", "High"]:
    mask = df_risk["risk_group"] == group

    kmf.fit(
        df_risk.loc[mask, "time"],
        event_observed=df_risk.loc[mask, "event"],
        label=f"{group} Risk (n={mask.sum()})"
    )

```

```

    )

    kmf.plot_survival_function()

plt.title("Kaplan–Meier Curves by Cox Model Risk Groups", fontsize=16)

plt.xlabel("Time (months)", fontsize=14)

plt.ylabel("Survival Probability", fontsize=14)

plt.grid(True, alpha=0.3)

plt.legend(fontsize=12)

plt.show()


# === Creating Ensemble Risk Score Using Subset RSF ===

print("=== Step 4.6: Creating Ensemble Risk Score (Cox + RSF using subset) ===")

from sklearn.preprocessing import MinMaxScaler

# -----

# 1. Cox risk (full dataset)

# -----

cox_risk = cox_pipeline.predict(X_test)

# -----

# 2. RSF predicted risk

# -----

subset_size = 5000

idx = np.random.choice(len(X_test_rsf), subset_size, replace=False)

X_rsf_subset = X_test_rsf.iloc[idx]

rsf_risk_subset = rsf.predict(X_rsf_subset)

# Prepare RSF risk for *all* X_test by interpolation

# Assign subset predictions and fill others with median

rsf_risk = np.full(len(X_test_rsf), np.median(rsf_risk_subset))

rsf_risk[idx] = rsf_risk_subset

```

```

# -----

# 3. Normalize both risk scores to 0–1

# -----

scaler = MinMaxScaler()

df_risks = pd.DataFrame({

    "cox": cox_risk,

    "rsf": rsf_risk

})

df_risks[["cox_norm", "rsf_norm"]] = scaler.fit_transform(

    df_risks[["cox", "rsf"]]

)

# -----

# 4. Ensemble = average of normalized cox + rsf

# -----

df_risks["ensemble_risk"] = 0.5 * df_risks["cox_norm"] + 0.5 * df_risks["rsf_norm"]

# -----

# 5. Add survival labels

# -----

df_risks["time"] = y_test["time"]

df_risks["event"] = y_test["event"]

# -----

# 6. Percentile stratification: Low / Medium / High risk

# -----

df_risks["ensemble_group"] = pd.qcut(

    df_risks["ensemble_risk"],

    q=3,

    labels=["Low", "Medium", "High"]

)

```

```

print(df_risks.head())

print("\nEnsemble risk computation complete (subset-based RSF).")

# === Kaplan–Meier Curves for Ensemble Risk Groups ===

kmf = KaplanMeierFitter()

plt.figure(figsize=(12, 7))

# Order groups Low → Medium → High for clean legend and interpretation
for group in ["Low", "Medium", "High"]:
    mask = df_risks["ensemble_group"] == group
    n = mask.sum()
    kmf.fit(
        df_risks.loc[mask, "time"],
        event_observed=df_risks.loc[mask, "event"],
        label=f"{group} Risk (n={n})"
    )
    kmf.plot(ci_show=False, linewidth=2)

plt.title("Kaplan–Meier Survival Curves by Ensemble Risk Group", fontsize=16)
plt.xlabel("Time (months)")
plt.ylabel("Survival Probability")
plt.grid(True, alpha=0.3)
plt.show()

```

```

# =====

# Prepare Data for DeepSurv

# =====

import torch

from sklearn.preprocessing import LabelEncoder, StandardScaler

print("== Step 5.1: Preparing data for DeepSurv ==")

# -----

# 1. Copy original dataframes

# -----

X_train_ds = X_train.copy()

X_test_ds = X_test.copy()

# Identify feature types

num_cols = X_train_ds.select_dtypes(include=['int64', 'float64']).columns.tolist()

cat_cols = X_train_ds.select_dtypes(include=['object', 'category']).columns.tolist()

print(f'Numeric columns: {len(num_cols)}')

print(f'Categorical columns: {len(cat_cols)}')

# Dictionary to store encoders

label_encoders = {}

# -----

# 2. Label encode categorical features SAFELY

# -----

for col in cat_cols:

    le = LabelEncoder()

    # Convert values to string

    X_train_ds[col] = X_train_ds[col].astype(str)

    X_test_ds[col] = X_test_ds[col].astype(str)

```

```

# Replace unseen labels in TEST with "Unknown"

X_test_ds[col] = X_test_ds[col].where(

    X_test_ds[col].isin(X_train_ds[col].unique()),

    "Unknown"

)

# Ensure training set contains "Unknown" if test has it

if "Unknown" not in X_train_ds[col].unique():

    X_train_ds.loc[:, col] = X_train_ds[col].replace(np.nan, "Unknown")


# Fit on TRAIN classes + "Unknown"

le.fit(np.append(X_train_ds[col].unique(), ["Unknown"]))


# Transform both sets

X_train_ds[col] = le.transform(X_train_ds[col])

X_test_ds[col] = le.transform(X_test_ds[col])

label_encoders[col] = le

print("Categorical encoding complete.")


# -----

# 3. Scale numeric columns

# -----

scaler = StandardScaler()

X_train_ds[num_cols] = scaler.fit_transform(X_train_ds[num_cols])

X_test_ds[num_cols] = scaler.transform(X_test_ds[num_cols])

print("Scaling complete.")


# -----

# 4. Convert to numpy arrays for DeepSurv

```

```

# -----

X_train_np = X_train_ds.values.astype(np.float32)
X_test_np = X_test_ds.values.astype(np.float32)
print("Converted to NumPy arrays.")

# -----

# 5. Convert survival labels to tensors
# -----

# y_train and y_test are structured arrays → convert manually
time_train_ds = torch.tensor(y_train["time"].astype(np.float32))
event_train_ds = torch.tensor(y_train["event"].astype(np.float32))
time_test_ds = torch.tensor(y_test["time"].astype(np.float32))
event_test_ds = torch.tensor(y_test["event"].astype(np.float32))
print("Labels converted to tensors.")

# -----

# 6. Final prints
# -----

print("Final training data shape:", X_train_np.shape)
print("Final test data shape:", X_test_np.shape)
print("Input dimension for DeepSurv model:", X_train_np.shape[1])

# =====

# Build DeepSurv Model
# =====

import torch
import torch.nn as nn

```

```

input_dim = X_train_np.shape[1]

class DeepSurv(nn.Module):

    def __init__(self, input_dim):
        super(DeepSurv, self).__init__()

        self.net = nn.Sequential(

            nn.Linear(input_dim, 128),

            nn.ReLU(),

            nn.Dropout(0.2),

            nn.Linear(128, 64),

            nn.ReLU(),

            nn.Dropout(0.2),

            nn.Linear(64, 1)

        )

    def forward(self, x):

        return self.net(x)

deepsurv = DeepSurv(input_dim)

print(deepsurv)

print("\nDeepSurv model successfully created.")


import gc

import sys

# Print all variables in the environment

for var_name in dir():

    if not var_name.startswith("_"):

        try:

            var_value = eval(var_name)

```

```

        print(f"{var_name}: {type(var_value)}")
    except:
        pass

import os

import torch

# Disable Inductor (root cause of circular import)
os.environ["TORCHINDUCTOR"] = "0"
os.environ["TORCHINDUCTOR_DISABLED"] = "1"
os.environ["PYTORCHINDUCTOR"] = "0"
os.environ["TORCH_LOGS"] = ""

# Disable Dynamo JIT
import torch._dynamo
torch._dynamo.disable()
torch._dynamo.config.suppress_errors = True

# Disable Torch Compile (PyTorch 2.0+)
if hasattr(torch, "compile"):
    torch.compile = lambda *args, **kwargs: args[0]

print("Torch Inductor & Dynamo disabled successfully")

# =====

# Train DeepSurv

# =====

from torch.utils.data import DataLoader, TensorDataset

if not hasattr(torch, "version"):

    class DummyVersion:

        hip = False

    torch.version = DummyVersion()

```

```

# Disable Torch Dynamo / Inductor (avoids compilation crashes)

import torch._dynamo

torch._dynamo.config.suppress_errors = True

torch._dynamo.disable()


# -----

# Device

# -----

device = torch.device("cpu") # Your machine uses CPU

print("Using:", device)


# -----

# Convert processed training data to tensors

# (X_train_ds and X_test_ds come from Step 5.1)

# -----

X_train_t = torch.tensor(X_train_ds.values, dtype=torch.float32).to(device)

X_test_t = torch.tensor(X_test_ds.values, dtype=torch.float32).to(device)

time_train_t = time_train_ds.to(device)

event_train_t = event_train_ds.to(device)


# -----

# Build dataset + dataloader

# -----

train_dataset = TensorDataset(X_train_t, time_train_t, event_train_t)

train_loader = DataLoader(train_dataset, batch_size=2048, shuffle=True)


# -----

```

```

# Optimizer (NOW SAFE)

# -----

optimizer = torch.optim.Adam(deepsurv.parameters(), lr=1e-3, weight_decay=1e-4)

# -----

# Cox Partial Likelihood Loss (SAFE VERSION)

# -----

def negative_log_likelihood(risk_pred, time, event):

    # exp(risk)

    hazard_ratio = torch.exp(risk_pred)

    # Reverse safely

    reversed_hr = torch.flip(hazard_ratio, dims=[0])

    # cumulative sum on reversed array, then flip back

    cum_sum = torch.flip(torch.cumsum(reversed_hr, dim=0), dims=[0])

    log_risk = torch.log(cum_sum + 1e-8)

    # Cox partial likelihood

    uncensored = risk_pred - log_risk

    loss = -torch.mean(uncensored[event == 1])

    return loss

# -----

# Training Loop

# -----

epochs = 20

deepsurv.train()

for epoch in range(1, epochs + 1):

    total_loss = 0.0

    for batch_x, batch_time, batch_event in train_loader:

```

```

optimizer.zero_grad()

pred = deepsurv(batch_x).squeeze()

loss = negative_log_likelihood(pred, batch_time, batch_event)

loss.backward()

optimizer.step()

total_loss += loss.item()

print(f'Epoch {epoch}/{epochs} - Loss: {total_loss:.4f}')

print("Training complete.")

# =====

# Evaluate DeepSurv

# =====

from sksurv.metrics import concordance_index_censored

# Switch model to evaluation mode

deepsurv.eval()

with torch.no_grad():

    pred_test = deepsurv(X_test_t).squeeze().cpu().numpy()

# DeepSurv risk = linear predictor (higher = worse)

risk_scores = pred_test

event_test_np = event_test_ds.cpu().numpy().astype(bool)

time_test_np = time_test_ds.cpu().numpy()

c_index = concordance_index_censored(

    event_test_np,

    time_test_np,

    risk_scores

)[0]

```

```

print(f'DeepSurv Test C-index: {c_index:.4f}')

# =====

# DeepSurv Survival Curves

# =====

deepsurv.eval()

with torch.no_grad():

    pred_test = deepsurv(X_test_t).squeeze().cpu().numpy()

risk_scores = pred_test

# Create a tidy DataFrame

df_deep = pd.DataFrame({

    "risk": risk_scores,

    "time": time_test_np,

    "event": event_test_np

})

# Assign risk groups using percentiles (33% / 66%)

df_deep["risk_group"] = pd.qcut(df_deep["risk"], q=3, labels=["Low", "Medium", "High"])

# Plot

kmf = KaplanMeierFitter()

plt.figure(figsize=(10, 7))

for group in ["Low", "Medium", "High"]:

    mask = df_deep["risk_group"] == group

    kmf.fit(df_deep.loc[mask, "time"],

            event_observed=df_deep.loc[mask, "event"],

```

```

        label=f'{group} Risk")

    kmf.plot(ci_show=False)

plt.title("DeepSurv — Survival Curves by Risk Group")

plt.xlabel("Time (Months)")

plt.ylabel("Survival Probability")

plt.grid(True, alpha=0.3)

plt.show()


# === Prepare a smaller dataset for SHAP ===

import shap

print("Preparing SHAP dataset...")

# Use the correct variables from your workspace

X_test_tensor = X_test_t      # shape: [88278, 22]

time_test_tensor = time_test_ds # tensor

event_test_tensor = event_test_ds # tensor


# Pick 2000 samples safely

shap_size = 2000

indices = np.random.choice(X_test_tensor.shape[0], shap_size, replace=False)


X_shap = X_test_tensor[indices]

time_shap = time_test_tensor[indices]

event_shap = event_test_tensor[indices]


print("SHAP dataset shape:", X_shap.shape)

print("Time shape:", time_shap.shape)

print("Event shape:", event_shap.shape)

```

```

# === Build SHAP Explainer for DeepSurv ===

print("Creating SHAP explainer ")

# Put model in evaluation mode

deepsurv.eval()

# Select a small background dataset (required for DeepExplainer)

background_size = 100

background_idx = np.random.choice(X_test_tensor.shape[0], background_size, replace=False)

background = X_test_tensor[background_idx]

# Convert to float32 (required by SHAP + PyTorch)

background = background.to(torch.float32)

X_shap_float = X_shap.to(torch.float32)

# Create the DeepExplainer

explainer = shap.DeepExplainer(deepsurv.net, background)

print("Computing SHAP values...")

shap_values = explainer.shap_values(X_shap_float)

print("SHAP values computed successfully!")

print("SHAP values shape:", np.array(shap_values).shape)


# === SHAP Summary Plot ===

# Create list of feature names for DeepSurv

feature_names = num_cols + cat_cols

print("Number of feature names:", len(feature_names))

# Convert tensor -> numpy for SHAP summary plot

X_shap_np = X_shap.cpu().numpy()

plt.figure(figsize=(12, 14))

shap.summary_plot(

    shap_values,

    X_shap_np,

```

```

    feature_names=feature_names,

    plot_type="dot",

    max_display=len(feature_names), # ensure ALL features shown

    show=True

)

# === SHAP Bar Plot (Global Importance) ===

print("Generating SHAP Bar Plot...")

plt.figure(figsize=(10, 6))

shap.summary_plot(

    shap_values,

    X_shap_np,

    plot_type="bar",

    feature_names=feature_names,

    show=False

)

plt.title("DeepSurv — Global Feature Importance (SHAP Bar Plot)", fontsize=16)

plt.tight_layout()

plt.show()


# === SHAP Dependence Plots ===


top_features = ["year_dx", "primary_site", "ajcc_n", "tumor_size", "laterality"]

for f in top_features:

    plt.figure(figsize=(8, 6))

    shap.dependence_plot(

        f,

        shap_values,

        X_shap_np,

```

```

        feature_names=feature_names,

        show=False

    )

    plt.title(f"SHAP Dependence Plot — {f}", fontsize=15)

    plt.tight_layout()

    plt.show()

# --- Interaction Heatmap (Safe Approximation for DeepSurv) ---

print("Building interaction heatmap...")

# compute approximate interaction strengths

interaction_matrix = np.zeros((len(feature_names), len(feature_names)))

for i in range(len(feature_names)):

    for j in range(len(feature_names)):

        interaction_matrix[i, j] = np.corrcoef(

            shap_values[:, i],

            X_shap_np[:, j]

        )[0, 1]

plt.figure(figsize=(14, 12))

plt.imshow(np.abs(interaction_matrix), cmap='viridis')

plt.colorbar(label="Interaction Strength")

plt.xticks(range(len(feature_names)), feature_names, rotation=90)

plt.yticks(range(len(feature_names)), feature_names)

plt.title("DeepSurv Interaction Heatmap (Correlation-based SHAP Approximation)")

plt.tight_layout()

plt.show()

print("=== Ensemble Feature Importance (Aligned 22 Features, Manual RSF PI) ===")

```

```

from sksurv.linear_model import CoxPHSurvivalAnalysis

# -----

# 0. Common feature names (22 features, from DeepSurv data)

# -----

feature_names = list(X_train_ds.columns)

print(f"Number of features (should be 22): {len(feature_names)}")

# =====

# Cox PH Importance (refit on 22-feature dataset)

# =====

cox22 = CoxPHSurvivalAnalysis()

cox22.fit(X_train_ds, y_train)

cox_coef = cox22.coef_.flatten()    # shape: (22,)

cox_importance = np.abs(cox_coef)

cox_importance /= (cox_importance.sum() + 1e-8)

print("Cox importance ready.")

# =====

# RSF Importance — manual permutation on risk scores

# =====

print("Computing RSF permutation importance (manual) ...")

# ---- Choose a subset of test data for speed ----

subset_size = min(2000, len(X_test_rsf))

rng = np.random.RandomState(42)

idx = rng.choice(len(X_test_rsf), subset_size, replace=False)

# Align columns with feature_names

X_rsf_subset = X_test_rsf.iloc[idx][feature_names].copy()

```

```

# ---- baseline risk scores from RSF ----

# rsf.predict(...) in your case returns numeric risk scores (1D float array)

baseline_risk = rsf.predict(X_rsf_subset).astype(float) # shape: (subset_size,)

rsf_importances = []

# ---- for each feature, shuffle and measure impact on risk ----

for col in feature_names:

    X_shuffled = X_rsf_subset.copy()

    X_shuffled[col] = rng.permutation(X_shuffled[col].values)

    shuffled_risk = rsf.predict(X_shuffled).astype(float)

    diff = np.abs(baseline_risk - shuffled_risk)    # feature impact on risk

    rsf_importances.append(diff.mean())

rsf_importance = np.array(rsf_importances)

rsf_importance /= (rsf_importance.sum() + 1e-8)

print("RSF importance ready.")

# =====

# DeepSurv SHAP importance (already computed as shap_values)

# =====

# shap_values shape: (n_shap_samples, 22)

shap_importance = np.mean(np.abs(shap_values), axis=0)

shap_importance /= (shap_importance.sum() + 1e-8)

print("DeepSurv SHAP importance ready.")

# =====

# Sanity check: all arrays must have same length

# =====

print(

```

```

    "Lengths:",
    len(feature_names),
    len(cox_importance),
    len(rsf_importance),
    len(shap_importance),
)

# =====

# Build final ensemble feature-importance table

# =====

ensemble_df = pd.DataFrame({
    "feature": feature_names,
    "cox_importance": cox_importance,
    "rsf_importance": rsf_importance,
    "deepsurv_importance": shap_importance
})

ensemble_df["ensemble_mean"] = ensemble_df[
    ["cox_importance", "rsf_importance", "deepsurv_importance"]
].mean(axis=1)

ensemble_df = ensemble_df.sort_values("ensemble_mean", ascending=False).reset_index(drop=True)

print("\n==== Final Ensemble Feature Importance Table (Top 10) ===")

display(ensemble_df.head(10))

```